R4EwE:

August 25, 2026

Title R for Ecopath with Ecosim (R4EwE)

Version 0.0.0.9000

Description Utilities, workflows, and helpers for Ecopath with Ecosim (EwE) users: environmental driver extraction, running EwE from R, genetic-algorithm calibration, objective functions and likelihoods, and reading and plotting model output.

License MIT + file LICENSE

URL https://github.com/dchagaris/R4EwE

BugReports https://github.com/dchagaris/R4EwE/issues

Encoding UTF-8

Language en-US

Roxygen list(markdown = TRUE)

Imports colorRamps,
digest,
doParallel,
doSNOW,
fields,
foreach,
gifski,
httr,
maps,
ncdf4,
raster,
reshape2,
rvest,
stats,
terra,
viridis

Suggests aquamapsdata,
DBI,
dplyr,
knitr,
pkgbuild,
remotes,
rmarkdown,
RSQLite

Remotes raquamaps/aquamapsdata

VignetteBuilder knitr

Config/roxygen2/version 8.0.0

RoxygenNote 7.3.3

Contents

`.r4ewe_worker_exports` *Canonical list of objects to clusterExport for R4EwE GA/sensitivity workers.*

Description

Returns the character vector of object names that every worker process needs in its global environment to call `safe_runEwE` → `fn.runEwE` → any of the supported objective functions (`fn.objfxn1`, `fn.objfxn2`, `fn.ecospace_objfxn`). Use it in your top-level `clusterExport()` call so the three clusterExport sites (script, `ensure_cluster`, `fn.GA`'s every-5-gen reset) stay in sync.

Usage

```
.r4ewe_worker_exports()
```

Value

Character vector of object names.

archive_ga *Archive a completed GA calibration run*

Description

Minimum-viable phase-agnostic archive for a completed fn.GA run whose final-pop hook (preserve.final.output=TR has already written final_ga_pop<ts>.csv and final_ga_runs_<ts>.csv to dir.results.ga. Does four things:

1. reads the final-pop CSVs to identify the best-fit individual,
2. copies calibration script snapshots (and the R4EwE git rev) to phase<N>_<ts>/code/,
3. extracts absolute-value GUI-ready CSVs (vuls, disp, fleetdyn, env+redtide) from the best-fit parvec / cmd file and writes them to phase<N>_<ts>/gui_ready/,
4. writes a README.md summarising config, best LL, and known caveats.

Creates the phase-1 subdir structure (code/, results/, plots/, gui_ready/, logs/) for consistency; only code/ and gui_ready/ are populated. Bundle the rest by hand or extend with bundle_results/bundle_plots/bundle_logs in a future revision.

Usage

```
archive_ga(
  phase,
  timestamp = NULL,
  dir.results.ga = NULL,
  code_files = NULL,
  r4ewe_dir = "C:/dchagaris/GitHub/R4EwE",
  notes = NULL,
  overwrite = FALSE,
  move_results = TRUE
)
```

Arguments

phase	Integer phase number (1, 2, 3, ...). Becomes the archive-dir prefix (phase<N>_<timestamp>/) and the README title.
timestamp	GA-run timestamp string (matches the <ts> suffix on final_ga_pop<ts>.csv). Defaults to timestamp in .GlobalEnv.
dir.results.ga	Directory holding final_ga_pop<ts>.csv, final_ga_runs_<ts>.csv, and ga_results_<ts>.csv Defaults to dir.results.ga in .GlobalEnv.
code_files	Character vector of script paths to copy into code/. Relative paths are resolved against dirname(dir.results.ga). Missing files are skipped silently. Default = the standard four calibration scripts.
r4ewe_dir	Path to the R4EwE git working tree for commit-hash capture. Set to NULL to skip.
notes	Optional character vector of extra bullet lines appended to a trailing ## Additional notes section in the README. Use for phase-specific caveats.
overwrite	If FALSE (default) and the target archive dir already exists, error out. Set TRUE to reuse and overwrite.

`move_results` If `TRUE` (default), moves every timestamp-matching file out of `dir.results.ga` into the archive after reads complete: CSVs to `results/`, PDFs/PNGs to `plots/`. Set `FALSE` to leave the source files in `dir.results.ga` (copy semantics were the old phase-1 behavior).

Value

Invisibly returns the archive directory path.

`crossover` *One-point crossover for paired parents*

Description

Applies one-point crossover to pairs of parents to create offspring.

Usage

```
crossover(parents)
```

Arguments

`parents` Matrix of parent individuals (rows = individuals, columns = parameters).

Details

Processes rows in pairs (1–2, 3–4, ...). With probability 0.8, a single crossover point is drawn uniformly from positions 1 to n_pars - 1, and the trailing gene segments are swapped between the pair.

Requires global variables `pop_size` (number of rows in `parents`) and `n_pars` (number of columns) to be defined in the calling environment.

Value

Matrix of offspring with the same dimensions as `parents`.

`crossover_uniform` *Uniform crossover with grouped (linked) parameters*

Description

Performs uniform crossover on pairs of parents, but treats specified columns as linked "groups" that must be swapped together.

Usage

```
crossover_uniform(
  parents,
  group_id = 1:ncol(parents),
  p_cross = 0.8,
  p_group = 0.5
)
```

Arguments

parents	Numeric matrix. Rows = individuals, columns = parameters.
group_id	Vector of length ncol(parents) giving the group ID of each column. Columns with the same ID form a linkage block and will be swapped together. Use a unique ID per column to leave it ungrouped. NA IDs are treated as unique.
p_cross	Probability of attempting crossover for a parent pair.
p_group	Probability a group is swapped when crossover occurs. Either a scalar (applied to all groups) or a vector of length = number of groups.

Details

Pairs rows (1&2, 3&4, ...) and for each pair attempts crossover with probability p_cross. If crossover happens, a binary mask is drawn over groups with probability p_group, and all columns in the selected groups are swapped.

Value

Matrix of offspring with the same dimensions as parents.

---

fill_coastal_cells *Fill missing cells*

Description

Iteratively applies 3x3 nearest neighbor means to NA cells until all water cells have data. Typically used to fill areas near the coastlilne with grids don't align perfectly.

Usage

```
fill_coastal_cells(r, w = 3, max_iter = 50, mask = depth)
```

Arguments

r	A raster layer.
w	Neighborhood area, must be odd.
max_iter	Maximum number to times to iterate until cells are filled.
mask	Raster layer with land cells NA.

Value

Raster layer of dim(r), with all land cells NA and all water cells containing data.

Examples

```
## Not run:
fill_coastal_cells(r, mask=depth)

## End(Not run)
```

fn.akaike_weights *Akaike-style weights from a vector of LL values*

Description

Convenience helper to convert a vector of per-run LL (negative log-likelihood) values into Akaike-style weights for use with `fn.weighted_mrt_summary`. The weight for run `i` is `w_i = exp(-(LL_i - min(LL)) / scale)` normalized to sum to 1.

Usage

```
fn.akaike_weights(LL, scale = 2)
```

Arguments

`LL`	Numeric vector of LL values, one per run.
`scale`	Scaling factor in the exponent. Default 2, mimicking the standard AIC-weight convention that treats LL as $-2 \log L$. Increase `scale` to flatten the weights (broader ensemble); decrease it to concentrate on the very best fits. For a wide LL spread you may want `scale = 2 * df_eff` where `df_eff` is some effective degrees of freedom.

Value

Numeric vector of weights summing to 1 (same length as `LL`).

Examples

```
## Not run:
# From a GA fitness vector (final-gen min_LL = best individual)
w <- fn.akaike_weights(fitness_g20, scale = 2)
summary(w); sum(w)   # should sum to 1

## End(Not run)
```

fn.append_netcdf_glorys *Append two CMEMS GLORYS monthly NetCDF files along time*

Description

Appends a newer CMEMS GLORYS monthly NetCDF file to an older one along the `time` dimension and writes out a single merged NetCDF file.

The two input files are assumed to be identical in structure (variables, dimensions, metadata) and to differ only in their time coverage.

All data variables are appended automatically; coordinate variables (longitude, latitude, depth, time) are preserved from the original file.

Usage

```
fn.append_netcdf_glorys(old_file, new_file)
```

Arguments

old_file	Character string giving the file path to the older NetCDF file.
new_file	Character string giving the file path to the newer NetCDF file.

Details

The function performs basic compatibility checks on variable and dimension names. Data are appended along the time dimension, which is assumed to be the last dimension for all variables (as is standard for CMEMS GLORYS monthly products).

Variables are processed one at a time to limit memory usage.

Value

Invisibly returns the path to the output file (out_file).

See Also

nc_open, nc_create, abind

Examples

```
## Not run:
append_glorys_monthly_nc(
  old_file = "glorys_monthly_1993_2015.nc",
  new_file = "glorys_monthly_2016_2024.nc",
  out_file = "glorys_monthly_1993_2024.nc"
)

## End(Not run)
```

---

fn.connect_aquamaps_db

Connect to the AquaMaps SQLite database.

Description

Points the **aquamapsdata** connection cache at an AquaMaps SQLite database so that downstream queries (am_search_exact(), am_hspen()) read from the chosen file. The **aquamapsdata** package hard-codes its database location, so a custom db_path is supported by connecting directly and injecting the connection into the package's internal cache.

Usage

```
fn.connect_aquamaps_db(db_path = NULL)
```

Arguments

db_path	Optional path to an am.db SQLite file. If NULL (default) the package's standard location is used (and an informative error is raised, pointing to aquamapsdata::download_db(), if it is missing).

Details

The database is ~2 GB. Use aquamapsdata::download_db() once to fetch it to the default location, or pass db_path to use a copy kept elsewhere (e.g. a shared drive) and avoid re-downloading.

Value

Invisibly, the path of the database that was connected.

---

fn.disp_testval_tags *Make dispersal parameter taglines for sensitivity runs.*

Description

Create parameter lines for the command file to test sensitivity of dispersal rates.

Usage

```
fn.disp_testval_tags(disp.base, pct.change = 0.5)
```

Arguments

disp.base	A dataframe with baseline dispersal parameters, exported from EwE and read and renamed in the setup file.
pct.change	The percent change to use, applied as pct.change x base and (1+pct.change) x base. Default is 0.5 (+/-50%).

Value

A character vector containing the parameter taglines for the command file.

Examples

```
# example code:
## Not run: tags.vul <- fn.vul_testval_tags(predprey=data.frame(pred=1:5,prey=6:10, basevul=2), maxvul=1000,
```

fn.download_cefi_full_nc

Download full CEFI netcdf files

Description

Downloads the full netcdf for MOM6-COBALT variables from https://downloads.psl.noaa.gov/Projects/CEFI/regional_n
Uses the httr::GET function.

Usage

```
fn.download_cefi_full_nc(
  vars.cefi = vars.cefi,
  dir.cefi = dir.cefi,
  reg = "northwest_atlantic"
)
```

Arguments

vars.cefi	A dataframe returned by fn.get_cefi_vars(), likely subset to fewer variables. Must contain the variables "cefi_filename" and "cefi_rel_path".
dir.cefi	Parent directory for CEFI data, where the netcdf files will be saved.
reg	CEFI region, either northwest_atlantic or northeast_pacific

Examples

```
# example code:
## Not run: vars.cefi <- fn.get_cefi_vars(dir.cefi=dir.cefi, file.cefivars='CEFI_vars.csv', region='nwa')
```

fn.ecosim_plot_fits

Multipanel Ecosim prediction-vs-observation plots, single combined PDF.

Description

Reads predicted biomass, catch, landings (group-aggregated and fleet x group), and fishing mortality from one or more Ecosim output folders, overlays the matching observed timeseries from fn.read_ecosim_timeseries, and writes all panels to a single PDF (or the active device). For relative (non-absolute) observation series a scalar q = mean(obs[overlap]) / mean(pred[overlap, scale2run]) is applied so the obs lie on the prediction scale; absolute series are plotted as-is.

Usage

```
fn.ecosim_plot_fits(
  dir.out,
  obs.ts,
  vars = c("biomass", "catch", "landings", "discards", "F"),
  timestep = "annual",
  groups = "with_obs",
```

```
  scale2run = 1,
  plt.dims = c(3, 3),
  plt.cols = NULL,
  sim.labels = NULL,
  group.names = NULL,
  fleet.names = NULL,
  plot2pdf = FALSE,
  dir.plts = dir.out[1],
  run.label = Sys.Date()
)
```

Arguments

dir.out	Path to an Ecosim run output folder (containing the required csvs directly), or a parent folder containing one subfolder per run. Files are located with list.files(..., recursive = TRUE) so subfolders are picked up automatically and become colored lines per panel. May be a vector of folders.
obs.ts	Output of fn.read_ecosim_timeseries.
vars	Which panels to draw. Any subset of c("biomass","catch","landings","discards","F"). Default is all five. When "discards" is included and the run folder contains discardmortalityfleetgroup_*.csv and discardsurvivalfleetgroup_*.csv, three additional sections are produced: dead-discards (group, fleets stacked), surviving-discards (group, fleets stacked), and a combined dead-vs-surviving view. Observed dead and surviving are derived from the type-13 (Discards) and type-11 (DiscardMortality) timeseries in obs.ts.
timestep	"annual" or "monthly".
groups	One of "with_obs" (default; only groups with at least one matching obs series), "all" (every group in the pred file), or an integer vector of group pool codes.
scale2run	Which run column is used as the reference for computing the obs scalar q. Default 1.
plt.dims	Panel grid as c(rows, cols).
plt.cols	Vector of line colors for each run. Default seq_len(nruns).
sim.labels	Legend labels for runs. Default = run subfolder basenames.
group.names	Optional character vector indexed by group pool code; used in panel titles when provided. Falls back to "Group <n>".
fleet.names	Optional character vector indexed by fleet pool code; used in fleet x group panel titles and in the fleet legend on the F stacked-bar panels. Falls back to "F<f>".
plot2pdf	Logical. If TRUE, opens a PDF at file.path(dir.plts, "ecosim fits_<run.label>.pdf") and writes all panels to that single file.
dir.plts	Directory to write the PDF. Defaults to the first entry of dir.out.
run.label	String appended to the PDF filename. Defaults to today's date.

Value

Invisibly returns a list of the predicted arrays used (one entry per variable plotted).

Examples

```
## Not run:
ts <- fn.read_ecosim_timeseries("ts_mice_v4_discards.csv")
fn.ecosim_plot_fits(dir.out = "ecosim_sim00_init", obs.ts = ts, plot2pdf = TRUE)

## End(Not run)
```

`fn.ecosim_plot_ts` *Plot Ecosim predicted timeseries.*

Description

Makes a multipanel plot of 1 or more ecopace runs with observed data where available.

Usage

```
fn.ecosim_plot_ts(
  predB = predB,
  predC = predC,
  timestep = "annual",
  obs.ts = obs.ts,
  scale2run = 1,
  pltB.dims = c(3, 3),
  pltC.dims = c(3, 3),
  scaleCatch = FALSE,
  plt.cols = 1:dim(predB)[3],
  plt.obs = TRUE,
  sim.labels = NULL,
  dir.plts = dir.pred,
  plot2pdf = FALSE,
  run.label = Sys.Date()
)
```

Arguments

`predB`	An array of biomass predictions, as produced by fn.ecospace_ts2array()
`predC`	An array of catch predictions, as produced by fn.ecospace_ts2array()
`timestep`	Read 'annual' or 'monthly' data.
`obs.ts`	A list object containing reference timeseries information, as that created by fn.read_ecosim_timeseries().
`scale2run`	If multiple models, which to scale the observed values to.
`pltB.dims`	Numeric vector of length 2 specifying the number of rows and columns for multipanel plotting.
`pltC.dims`	Numeric vector of length 2 specifying the number of rows and columns for multipanel plotting.
`scaleCatch`	Logical. TRUE will scale the predictions so the mean is equal to that of the observed. Default is FALSE.
`plt.cols`	Vector of line colors.

plt.obs	Logical. Should observed data in obs.ts be plotted?
sim.labels	Optional character vector of labels for the runs, used in the legend. NULL (default) uses generic run numbers.
dir.plts	Location to save pdf plots. Defaults to dir.pred.
plot2pdf	Logical. TRUE (default) will save files to the specified directory.
run.label	Label appended to the output pdf file name (and used to tag the run). Defaults to the current date.

Value

Generate a figure in the active plotting device or saves as pdf file.

Examples

```
# example code:
## Not run: result <- fn.runEwE.parallel(runlist=myrunlist, obj.fxn=1, cl.export=list('myrunlist','obs.ts'))
```

---

fn.ecosim_predB_ts2array

Read Ecosim predicted biomass timeseries output.

Description

Reads biomass timeseries output csv into an array.

Usage

```
fn.ecosim_predB_ts2array(dir.out = dir.pred, timestep = "annual")
```

Arguments

dir.out	Output directory. All .csv biomass files created with Ecospace naming conventions in this directory will be read. Can be a vector of directories.
timestep	Read 'annual' or 'monthly' data.

Value

An array of biomass output with dimensions (nyrs,ngrps,nregions,nruns) #examples #example code: predB <- fn.ecospace_predB_ts2array (dir.out="./output/", timestep='annual',n.reg=0)

`fn.ecosim_predC_ts2array` *Read Ecosim predicted catch timeseries output.*

Description

Reads catch timeseries output csv, sums over fleets for each species, and puts into an array.

Usage

```
fn.ecosim_predC_ts2array(dir.out = dir.out, timestep = "annual")
```

Arguments

`dir.out`	Output directory. All .csv catch files created with Ecospace naming conventions in this directory will be read. Can be a vector of directories.
`timestep`	Read 'annual' or 'monthly' data.

Value

An array of catch output with dimensions (nyrs,ngrps,nregions,nruns) #examples #example code: predB <- fn.ecospace_predB_ts2array (dir.out="./output/", timestep='annual',n.reg=0)

`fn.ecospace_ascii2stack` *Read Ecospace output ascii files into a raster stack.*

Description

This function reads a subset of asc grids produced by an Ecospace run into a raster stack. Requires the terra package for faster processing.

Usage

```
fn.ecospace_ascii2stack(
  dir.out = dir.pred,
  do.vars = c("biomass"),
  do.grps = 1:12,
  do.fleets = 1:6,
  do.months = 1,
  do.years = 2010:2019,
  annualmaps = TRUE
)
```

Arguments

dir.out	The run output folder containing. It should contain a folder named 'asc', which holds all the output maps.
do.vars	The model output variables to read. Must match the naming convention of the output files, all lowercase. Current options are 'biomass','catch', and 'effort'.
do.grps	The model groups for which to read data.
do.fleets	The fleet numbers for which to read effort data.
do.months	The months to read output data, only used if monthly maps are output.
do.years	Years to read output data, will combine with do.months
annualmaps	Import annual (TRUE, default) or monthly (FALSE) maps from Ecospace.

Value

A SpatRaster stack from the terra package.

Examples

```
# example code:
## Not run: predB.maps <- fn.ecospace_ascii2stack(dir.out=dir.pred, do.vars=c('biomass'), do.grps=c(3,5,6,8,1
```

---

fn.ecospace_objfxn *Ecospace objective function: timeseries with fleet-specific landings and discards.*

Description

Composite lognormal cost function that fits Ecospace predictions to observed timeseries in `obs.ts` (as produced by `fn.read_ecosim_timeseries`). Supports biomass (type 0, 1), aggregated catch (6, 61), fleet-specific landings (12), fleet-specific discards (13, treated as TOTAL discards), and group-level total discards (19, 20). Region-aware: each obs series is matched to the prediction for its declared region from the `Region` header row in `obs.ts`. Reuses the shared prediction readers `fn.read_pred_ecospace_wide` and `fn.read_pred_ecospace_discards_split` so cost and plots share matching logic.

Usage

```
fn.ecospace_objfxn(
  dir.pred,
  obs.ts,
  group.names,
  fleet.names = NULL,
  run.idx = 1,
  regions = NULL,
  fit.abs.catch = TRUE,
  types = NULL,
  eps = NULL,
 spatial.obs = if (exists("spatial.obs", envir = .GlobalEnv)) get("spatial.obs", envir =
    .GlobalEnv) else NULL,
  spatial.weight = if (exists("spatial.weight", envir = .GlobalEnv))
```

```
    get("spatial.weight", envir = .GlobalEnv) else 1,
  model_styear = if (exists("model_styear", envir = .GlobalEnv)) get("model_styear",
    envir = .GlobalEnv) else 1985,
  region.areas = if (exists("region.areas", envir = .GlobalEnv)) get("region.areas",
    envir = .GlobalEnv) else NULL,
  return = c("both", "vector", "detail")
)
```

Arguments

dir.pred	Full path (or vector of paths) to Ecospace run folder(s) containing per-region Ecospace_Annual_Average_Region_<n>_<Var>.csv outputs.
obs.ts	A list as produced by fn.read_ecosim_timeseries (must include ts.head with Type, Poolcode, Poolcode2, Region, Weight, Absolute).
group.names	Character vector indexed by group pool code.
fleet.names	Character vector indexed by fleet pool code. Required to fit type-12/13 fleet-specific series; without it those series are skipped.
run.idx	Integer; which prediction run to evaluate when multiple runs are present. Default 1.
regions	Integer vector of region IDs to consider. NULL (default) auto-detects all Region_<n>_Biomass.csv files under dir.pred.
fit.abs.catch	Logical. If TRUE (default) absolute series are fit on their literal scale (q = 1); only relative series are q-rescaled. If FALSE, every series is q-rescaled by q = mean(pred[overlap]) / mean(obs[overlap]).
types	Optional integer vector of obs type codes to fit. Default fits every supported type present in obs.ts$ts.head$Type.
eps	Small floor applied to predictions before log() to avoid log(0). Default .Machine$double.eps^0.5
spatial.obs	Optional list of observed spatial maps (per group/pool) added as a spatial cross-entropy term via fn.spatial_LL. Defaults to spatial.obs in the global environment, or NULL to skip the spatial term.
spatial.weight	Scalar weight (prior precision) on the spatial likelihood term. Defaults to spatial.weight in the global environment, or 1.
model_styear	First model year, used to align predicted maps with the year window of each observed map. Defaults to model_styear in the global environment, or 1985.
region.areas	Named numeric vector of region areas (km^2), names = region ID as character; see fn.read_region_areas. Used to combine a series that spans several regions as an area-weighted mean of the per-region densities. Defaults to region.areas in the global environment, or NULL – in which case regions are combined with equal weights (with a warning), which over-weights small regions.
return	One of "both" (default), "vector", or "detail". "both" returns list(LL = vector, detail = data.frame); "vector" returns just the per-group cost vector (drop-in for legacy fn.objfxn1 callers / tight GA worker loops); "detail" returns just the per-series data frame. The detail data frame is obs.ts$ts.head (Title, Weight, Poolcode, Poolcode2, Type, Region, Type.name, Absolute, Reference, PoolRole) for the subset of series that were considered, with the computed columns q, n, obs.cv, obs.sd, loglik appended.

Value

The shape selected by `return`. Series with `Weight == 0` are recorded in the detail table but contribute zero to the cost.

`fn.ecospace_plot_fits` *Multipanel Ecospace prediction-vs-observation plots, single combined PDF.*

Description

Reads predicted biomass, catch, landings, discards (catch - landings), and an approximate fishing mortality (catch / biomass) from one or more Ecospace run folders for each region present, overlays matching observed timeseries from `fn.read_ecosim_timeseries` on a per-region basis (using the Region row in the obs header to assign each obs series to a region), and writes all panels to a single combined PDF. Each panel shows one group with one predicted line per region; observed points are colored to match the prediction line of the region the obs is labeled with. For relative (non-absolute) series, obs are scaled to the prediction of their own region by `q = mean(obs[overlap]) / mean(pred[overlap, obs.region])`.

Usage

```
fn.ecospace_plot_fits(
  dir.out,
  obs.ts,
  vars = c("biomass", "catch", "landings", "discards", "F"),
  timestep = "annual",
  regions = NULL,
  groups = "with_obs",
  styear = NULL,
  scale2run = 1,
  overlay = c("region", "run"),
  plt.dims = c(3, 3),
  region.colors = NULL,
  region.names = NULL,
  run.colors = NULL,
  sim.labels = NULL,
  group.names = NULL,
  fleet.names = NULL,
  scale.abs = FALSE,
  plot2pdf = FALSE,
  dir.plts = dir.out[1],
  run.label = Sys.Date(),
 region.areas = if (exists("region.areas", envir = .GlobalEnv)) get("region.areas",
    envir = .GlobalEnv) else NULL
)
```

Arguments

`dir.out`	Path to an Ecospace run folder containing the `Ecospace_Annual_Average_Region_<n>_<Var>.csv` (or `Ecospace_Average_*` for monthly) outputs, or a parent folder containing multiple run subfolders. May be a vector of folders.

`obs.ts` Output of `fn.read_ecosim_timeseries`. Must include the optional `Region` column (auto-populated as `NA` when the ts file has no Region header row); obs series without a matching region are dropped from the overlays.

`vars` Which panels to draw. Any subset of `c("biomass","catch","landings","discards","F")`. Default is all five. For `catch`, `landings`, and `discards` three views are produced: (1) a group-level section with one predicted line per region (obs overlaid per region), (2) an Ecosim-style stacked-bar section (one panel per group, fleets stacked, repeated per region, obs total overlaid), and (3) an individual fleet x group section (one panel per `fleet|group` column, one line per region, obs overlaid where the ts has a matching fleet+group series: type 12 for landings, type 13 for discards; absolute catch type 6/61 has no fleet dimension so those panels show predictions only). `discards` produces three group-level sections derived at fleet x group resolution and summed over fleets: **dead** discards `DD = catch - landings` (in EwE the Catch output is landings + dead discards, so the difference is the dead portion), **total** discards `DT = DD / Dmort` using the per-fleet type-11 DiscardMortality series from `obs.ts` (rule (a): a fleet with no type-11 entry uses Dmort = 1, i.e. DT = DD), and **surviving** discards `DS = DT - DD`. Total discards are overlaid against the type-19/20 DiscardsTotal obs (apples-to-apples); dead and surviving have no obs overlay (predictions only, shown for the same groups). Requires `fleet.names` to map the Catch/Landings `fleet|group` columns to fleet pool codes; without it every fleet defaults to Dmort = 1. `F` is `catch / biomass` per region.

`timestep` `"annual"` or `"monthly"`.

`regions` Integer vector of region IDs to plot. `NULL` (default) auto-detects all `Region_<n>_Biomass.csv` files found under `dir.out`. With `overlay = "run"` this must resolve to a single region (pass e.g. `regions = 0`).

`groups` One of `"with_obs"` (default), `"all"`, or an integer vector of group pool codes.

`styear` Calendar start year. `NULL` (default) uses the earliest year in `rownames(obs.ts$ts)`.

`scale2run` Integer index of the run used as the obs-scaling baseline (1..nruns). When `overlay = "region"` this run is also the *sole* prediction drawn (one line per region); when `overlay = "run"` every run is drawn and this one is only the obs baseline. Default 1.

`overlay` What the colored lines on each panel represent. `"region"` (default, backward-compatible): one line per region for the single run `scale2run` — requires you to pick the run when multiple runs are present. `"run"`: one line per run for a single region (compare runs, like `fn.ecosim_plot_fits`) — requires `regions` to resolve to a single region. With one region and one run either mode draws the single series.

`plt.dims` Panel grid `c(rows, cols)`.

`region.colors` Vector of line/point colors per region (in detected-region order). Default is a MATLAB-like palette (`colorRamps::matlab.like`, jet-ramp fallback) sized to the number of regions so colors do not recycle past 8. Used when `overlay = "region"`.

`region.names` Optional character vector parallel to detected regions (sorted by region ID) used in the bottom-of-page legend. Default `"Region <n>"`.

`run.colors` Vector of line colors per run (in run order). Default is a MATLAB-like palette (`colorRamps::matlab.like`, jet-ramp fallback) sized to the number of runs so colors do not recycle past 8. Used when `overlay = "run"`.

`sim.labels` Multi-run labels (bottom-of-page legend when `overlay = "run"`). Default = run subfolder basenames.

group.names	Character vector indexed by group pool code. Required to map obs (which uses pool codes) to Ecospace output columns (which use group names). If NULL, obs overlay is skipped (predictions still plotted).
fleet.names	Character vector indexed by fleet pool code. Used by the discards sections to map the Catch/Landings fleet|group columns to fleet pool codes so the per-fleet type-11 DiscardMortality series can be applied. If NULL, every fleet defaults to Dmort = 1 (total discards collapse to dead discards).
scale.abs	Logical, default FALSE. Diagnostic toggle. When FALSE, absolute series (e.g. type 6 Catches, type 1 BiomassAbs) are plotted at their literal values (q = 1) while only relative series are rescaled to the prediction by q = mean(obs[overlap]) / mean(pred[overlap, region]). When TRUE, the same q rescaling is applied to *every* series including absolute ones, so all obs are forced onto the prediction scale. Useful for separating a true scale/units mismatch (obs and pred disagree in magnitude but the rescaled shape matches) from a genuine dynamics error (shape still disagrees after rescaling).
plot2pdf	Logical. If TRUE, writes file.path(dir.plts, "ecospace fits_<run.label>.pdf").
dir.plts	Directory to write the PDF.
run.label	String appended to the PDF filename. Defaults to today's date.
region.areas	Named numeric vector of region areas (km^2); see fn.read_region_areas. Series spanning several regions are combined as an area-weighted mean of the per-region densities, matching fn.ecospace_objfxn.

Value

Invisibly returns the list of prediction arrays used (each is [years, groups, regions, runs]).

Examples

```
## Not run:
ts <- fn.read_ecosim_timeseries("ts_mice_v4_discards_ecospace_regions.csv")
fn.ecospace_plot_fits(dir.out = "sp00_5min_init", obs.ts = ts,
                      group.names = mygroupnames, plot2pdf = TRUE)

## End(Not run)
```

---

fn.ecospace_plot_series_fits

One-panel-per-series Ecospace prediction-vs-observation plots.

Description

Obs-centric counterpart to fn.ecospace_plot_fits: draws one panel per observation series in obs.ts$ts.head with the matching predicted series overlaid, rather than one panel per group with every matching obs series stacked on top of each other. The predicted series is constructed on the fly by summing the appropriate prediction array over the pool codes (Poolcodes) and regions (Regions) that the obs row calls for, so multi-pool / multi-region surveys plot cleanly against the aggregated prediction they are fit to. Type routing:

- types 0, 1 (relative / absolute biomass) -> per-region Biomass, summed across the listed pool codes and regions;

- types 6, 61, -6 (catch) -> group-aggregated `Catch`;
- type 12 (landings) -> group-aggregated `Landings`, group taken from `Poolcode2`;
- type 13 (fleet-specific dead discards obs; matched against **total** discards predicted at `fleet|group` resolution – see the `[[project_mice_ts_discards]]` note) -> `disc_total_fg` column `<fleet_name>|<group_n`
- types 19, 20 (total-discards obs) -> group-aggregated `disc_total`;
- types 4, 104 (fishing mortality) -> `catch / biomass` per region.

Any other type is skipped with a message. For relative series, obs are rescaled to the baseline run's prediction by q = `mean(obs[overlap]) / mean(pred[overlap, scale2run])` (as in `fn.ecospace_plot_fits`); absolute series are plotted at their literal values unless `scale.abs = TRUE`.

Usage

```
fn.ecospace_plot_series_fits(
  dir.out,
  obs.ts,
  timestep = "annual",
  regions = NULL,
  styear = NULL,
  scale2run = 1,
  types = NULL,
  groups = "all",
  series = NULL,
  scale.abs = FALSE,
  group.names = NULL,
  fleet.names = NULL,
  output = c("pdf", "png", "device"),
  plt.dims = c(3, 3),
  dir.plts = dir.out[1],
  run.label = Sys.Date(),
  run.colors = NULL,
  sim.labels = NULL,
 region.areas = if (exists("region.areas", envir = .GlobalEnv)) get("region.areas",
    envir = .GlobalEnv) else NULL
)
```

Arguments

`dir.out`	Path to an Ecospace run folder (or vector of them).
`obs.ts`	Output of `fn.read_ecosim_timeseries`.
`timestep`	`"annual"` or `"monthly"`.
`regions`	Integer vector of region IDs to load. `NULL` (default) auto-detects.
`styear`	Calendar start year (defaults to earliest year in `obs.ts$ts`).
`scale2run`	Integer index of the run used as the obs-scaling baseline (1..nruns).
`types`	Optional integer vector of `Type` codes to restrict to (e.g. `c(0, 1)` for biomass only). `NULL` = all supported.
`groups`	Group filter. `"all"` (default) keeps every obs series whose group codes map to at least one prediction column; an integer vector keeps only obs whose group set intersects it. "Group codes" here means `Poolcodes` for most types and `Poolcode2` for types 12 and 13 (see the type routing above).

series	Optional integer vector of row indices into obs.ts$ts.head. When supplied, types and groups are ignored and exactly these rows are plotted.
scale.abs	Logical, default FALSE. When TRUE, the q rescaling is also applied to absolute series.
group.names	Character vector indexed by group pool code. Required so obs pool codes can be mapped to Ecospace prediction columns.
fleet.names	Character vector indexed by fleet pool code. Required for type-13 (fleet x group) obs to build the fleet|group column key.
output	One of "pdf" (single combined multi-page PDF at file.path(dir.plts, paste0("ecospace series fits_", run.label, ".pdf"))), "png" (one PNG per series under file.path(dir.plts, paste0("ecospace series fits_", run.label))), or "device" (draw to the currently open device).
plt.dims	Panel grid c(rows, cols) for the PDF and device outputs. Ignored for PNG.
dir.plts	Directory to write PDF / PNG output.
run.label	String appended to the PDF filename / PNG subfolder. Defaults to today's date.
run.colors	Vector of line colors per run (in run order). Default is a MATLAB-like palette.
sim.labels	Multi-run labels used in the run legend. Default = run subfolder basenames.
region.areas	Named numeric vector of region areas (km^2); see fn.read_region_areas. Series spanning several regions are combined as an area-weighted mean of the per-region densities, matching fn.ecospace_objfxn.

Value

Invisibly returns a data.frame (one row per plotted series) with columns row_idx, title, type, poolcodes, poolcode2, regions, n_obs, q, rss.

See Also

fn.ecospace_plot_fits for the group-centric layout.

Examples

```
## Not run:
ts <- fn.read_ecosim_timeseries("ts_mice_v4_discards_ecospace_regions.csv")
fn.ecospace_plot_series_fits(dir.out = "sp00_5min_init", obs.ts = ts,
                             group.names = mygroupnames, fleet.names = myfleetnames,
                             output = "pdf")

## End(Not run)
```

---

fn.ecospace_plot_ts *Plot Ecospace predicted timeseries.*

Description

Makes a multipanel plot of 1 or more ecopace runs with observed data where available.

Usage

```
fn.ecospace_plot_ts(
  predB = predB,
  predC = predC,
  timestep = "annual",
  obs.ts = obs.ts,
  scale2run = 1,
  pltB.dims = c(3, 3),
  pltC.dims = c(1, 1),
  scaleCatch = FALSE,
  plt.cols = 1:dim(predB)[3],
  plt.obs = TRUE,
  dir.plts = dir.pred,
  plot2pdf = TRUE,
  run.label = Sys.Date()
)
```

Arguments

predB	An array of biomass predictions, as produced by fn.ecospace_ts2array()
predC	An array of catch predictions, as produced by fn.ecospace_ts2array()
timestep	Read 'annual' or 'monthly' data.
obs.ts	A list object containing reference timeseries information, as that created by fn.read_ecosim_timeseries().
scale2run	If multiple models, which to scale the observed values to.
pltB.dims	Numeric vector of length 2 specifying the number of rows and columns for multipanel plotting.
pltC.dims	Numeric vector of length 2 specifying the number of rows and columns for multipanel plotting.
scaleCatch	Logical. TRUE will scale the predictions so the mean is equal to that of the observed. Default is FALSE.
plt.cols	Vector of line colors.
plt.obs	Logical. Should observed data in obs.ts be plotted?
dir.plts	Location to save pdf plots. Defaults to dir.pred.
plot2pdf	Logical. TRUE (default) will save files to the specified directory.
run.label	Label appended to the output pdf file name (and used to tag the run). Defaults to the current date.

Value

Generate a figure in the active plotting device or saves as pdf file.

Examples

```
# example code:
## Not run: result <- fn.runEwE.parallel(runlist=myrunlist, obj.fxn=1, cl.export=list('myrunlist','obs.ts'))
```

fn.ecospace_predB_ts2array

Read Ecospace predicted biomass timeseries output.

Description

Reads biomass timeseries output csv into an array.

Usage

```
fn.ecospace_predB_ts2array(dir.out = dir.pred, timestep = "annual", n.reg = 0)
```

Arguments

dir.out	Output directory. All .csv biomass files created with Ecospace naming conventions in this directory will be read.
timestep	Read 'annual' or 'monthly' data.
n.reg	If output for multiple regions, how many? Default to 0.

Value

An array of biomass output with dimensions (nyrs,ngrps,nregions,nruns) #examples #example code: predB <- fn.ecospace_predB_ts2array (dir.out="./output/", timestep='annual',n.reg=0)

fn.ecospace_predC_ts2array

Read Ecospace predicted catch timeseries output.

Description

Reads catch timeseries output csv, sums over fleets for each species, and puts into an array.

Usage

```
fn.ecospace_predC_ts2array(dir.out = dir.pred, timestep = "annual", n.reg = 0)
```

Arguments

dir.out	Output directory. All .csv catch files created with Ecospace naming conventions in this directory will be read.
timestep	Read 'annual' or 'monthly' data.
n.reg	If output for multiple regions, how many? Default to 0.

Value

An array of catch output with dimensions (nyrs,ngrps,nregions,nruns) #examples #example code: predB <- fn.ecospace_predB_ts2array (dir.out="./output/", timestep='annual',n.reg=0)

`fn.ecospace_spatial_pred` *Read predicted Ecospace spatial biomass averaged over target years.*

Description

For each requested pool code, locates the matching `EcospaceMap<variable>-<species>.csv` (species inferred via `group.names[pc]`), parses the stacked timesteps, extracts steps whose `Year` attribute (fractional-year of model time; e.g. 30.083 for step 361 of a monthly model starting at year 0) falls in the target year window, and averages the corresponding grids.

Usage

```
fn.ecospace_spatial_pred(
  dir.pred,
  pool_codes,
  group.names,
  styear = 2015,
  endyear = 2024,
  model_styear = 1985,
  variable = "Biomass",
  nrows = 66L,
  ncols = 78L
)
```

Arguments

`dir.pred`	Path to an Ecospace output folder (parent of the `csv/` subdir).
`pool_codes`	Integer vector of WFS MICE pool codes to read.
`group.names`	Character vector indexed by pool code.
`styear, endyear`	Calendar year window (inclusive) to average over. Converted to model-relative years via `model_styear`: model years run from `model_styear` at Ecospace step 1.
`model_styear`	Calendar year corresponding to Ecospace step 1. Default 1985 (WFS MICE convention).
`variable`	Character; part of the filename (default `"Biomass"`).
`nrows, ncols`	Grid dimensions. Defaults 66 x 78.

Value

Named list of numeric matrices (one per pool code); NA cells are NODATA. NULL entry for any pool code whose file was not found.

`fn.envresp_offval_tags` *Make environmental-response "off" parameter taglines for sensitivity runs.*

Description

Creates one sensitivity tagline per environmental response function that neutralizes that response by replacing its shape with a flat **linear** response. For foraging responses both linear endpoints are set to 1 (flat response of 1, no spatial preference). For mortality responses both linear endpoints are set to 0 (flat response of 0, no extra mortality). The original shape and parameters of the response are discarded in the emitted tag; `par3` and `par4` of the linear shape are written as 0 placeholders. Use this instead of `fn.envresp_testval_tags` when you want to evaluate the effect of each env response by turning it off (one run per response) rather than perturbing its parameters.

Usage

```
fn.envresp_offval_tags(env.base)
```

Arguments

`env.base`	A data frame with one row per env response function. Must include the EcoSpace response index in either `fxn_num` (preferred) or `resp.idx` (legacy), and `resp_type` (character; either `"foraging"` or `"mortality"`). Other columns are passed through unchanged so the result can be `rbind.fill`'d into a runlist alongside other parameter types.

Value

A data frame with one row per env response, with `shape.idx` set to 1 (linear), `par1..par4` overwritten with the off values, and a `tag` column containing the formatted `<ECOSPACE_ENVIRONMENTAL_RESPONSE_INDEXED>` tagline. Rows with unrecognized `resp_type` are dropped with a warning.

Examples

```
## Not run: tags.off <- fn.envresp_offval_tags(envpars.base)
```

`fn.envresp_testval_tags` *Make environmental response shape parameter taglines for sensitivity runs.*

Description

Create parameter lines for the command file to test sensitivity to environmental response shapes.

Usage

```
fn.envresp_testval_tags(env.base, fact = 2)
```

Arguments

env.base	A dataframe with baseline mediation parameters, this has to be created manually.
fact	Multiplicative factor used to widen and narrow each baseline response shape when generating the low/high sensitivity taglines. Default 2.

Value

A character vector containing the parameter taglines for the command file.

Examples

```
# example code:
## Not run: tags.vul <- fn.vul_testval_tags(predprey=data.frame(pred=1:5,prey=6:10, basevul=2), maxvul=1000,
```

---

fn.fg_meta *Parse Ecospace* fleet|group *column names.*

Description

Splits Ecospace "fleet|group" column-name strings into a data frame of fleet and group name parts. Use with the column dimnames of arrays returned by fn.read_pred_ecospace_wide to map columns to fleet and group pool codes.

Usage

```
fn.fg_meta(fg_names)
```

Arguments

fg_names	Character vector of "fleet|group" column names.

Value

A data frame with columns fg, fleet_nm, group_nm.

---

fn.GA *Run genetic algorithm calibration for Ecospace*

Description

Sets GA controls from myconfig, writes a results CSV, runs a base model, evaluates an initial population, then iterates generations with elitism, selection, crossover, mutation, and optional parameter-bound penalties. Progress and summary stats are printed each generation.

Usage

```
fn.GA(myconfig)
```

Arguments

`myconfig`	List of GA settings, expected fields:

- popSize: population size
- n_gen: number of generations
- pmutation: per-gene mutation probability
- elitism: number of elites kept each generation
- do.penalty: logical, apply penalties for bound violations
- pen.wt.mult: scalar multiplier for penalty weight
- gapop.dist: initial population distribution ("unif" or "ln")

Details

Uses helper functions: fn.GApop, fn.parvec2cmd, fn.runEwE.gapop, select_parents, crossover_uniform, mutate, and optionally ensure_cluster. Writes a CSV to dir.main using a timestamped filename, and cleans output directories as needed. Several values are read from the environment, such as est_par_vec, dir.main, timestamp, cmd_base, run_dir, U.bounds, L.bounds. Global variables pop_size, n_generations, mutation_rate, elitism, do.penalty, pen.wt.mult, and gapop.dist are set for use by downstream helpers.

Value

Invisibly returns NULL. Results (fitness summaries and best parameters) are appended to the CSV and printed to the console.

`fn.GApop` *Generate an initial GA population matrix*

Description

Creates the initial population for a genetic algorithm using either uniform draws within parameter bounds or (optionally) normal/gamma draws centered on existing parameter estimates. Uses several global objects created elsewhere.

Usage

```
fn.GApop(
  run_config = myconfig,
  pardist = "uniform",
  vuldist = "log-uniform",
  rtdist = "log-uniform",
  fltdist = "normal"
)
```

Arguments

`run_config`	List containing GA settings; must include `popSize`.
`pardist`	Character string; distribution type, either"uniform" (uniform between bounds) or "normal" (normal/gamma hybrid). Default "uniform"

vuldist — Character string; distribution type for vulnerability parameters. One of "log-uniform" (default — exp(runif(log(L), log(U))), evenly samples each order of magnitude so the sensitive low end gets equal density), "uniform" (linear uniform between bounds; over-weights the high end when bounds span multiple orders of magnitude), or "gamma" (shape=2, rate=2/base.val; mode near base/2, mass concentrated around the base value).

rtdist — Character string; distribution type for red tide parameters. Either "log-uniform" (default — symmetric in log space around the baseline of 1; appropriate when bounds span more than one order of magnitude) or "uniform" (same linear uniform as the other params, follows pardist).

fltdist — Character string; distribution type for fleet dynamics parameters (effective power and effort multiplier). One of "normal" (default — rnorm(mean = base.val, sd = fltdyn.cv * base.val) centered on each parameter's base value, then clipped to [L.bounds, U.bounds]) or "uniform" (linear uniform between the bounds, follows whatever pardist produced).

Details

When red tide parameters are present, the first 6 rows of the returned matrix are overwritten with explicit anchor combinations to guarantee corner-case coverage in the initial population:

1. baseline (all RT adj = 1)
2. gentle, early-onset (inflection.adj = min, slope.adj = min)
3. steep, late-onset (inflection.adj = max, slope.adj = max)
4. early crash (inflection.adj = min, slope.adj = max)
5. late & gradual (inflection.adj = max, slope.adj = min)
6. RT effectively off (slope.adj = min; inflection.adj as drawn)

Non-RT parameters in those rows keep their random draws so the anchors still vary on the rest of the parameter vector.

Value

A numeric matrix of dimension run_config$popSize x n_pars.

---

fn.gen_slim_copy *Slim-copy a generation of Ecospace GA run folders*

Description

Copies a directory of Ecospace GA run folders (e.g. gen30/) into a sibling output directory (default <dir.in>_small) that keeps only annual timeseries plus spatial biomass/catch maps for a chosen set of groups, and optionally effort maps for a chosen set of fleets. Folder and file names are preserved verbatim so downstream R4EwE readers work unchanged. Files are copied byte- for-byte (no re-parse).

Usage

```
fn.gen_slim_copy(
  dir.in,
  dir.out = paste0(dir.in, "_small"),
  groups = c("gag", "red grouper"),
  fleets = NULL,
  map.kinds = c("Biomass", "Catch"),
  keep.cmd = TRUE,
  keep.misc = TRUE,
  overwrite = FALSE,
  ncores = 1L,
  long.paths = TRUE,
  verbose = TRUE
)
```

Arguments

dir.in	Character. Directory of run folders (e.g. .../gen30).
dir.out	Character. Output directory. Default paste0(dir.in, "_small").
groups	Character vector of group name stems (matched against the EcospaceMap...-<stem> <stanza>.csv filename). Default gag + red grouper.
fleets	Character vector of exact fleet names for effort maps, or the string "all", or NULL (default) to drop every effort map.
map.kinds	Character vector of spatial map kinds to consider for the group filter. Default c("Biomass", "Catch"). Effort is controlled by fleets, not this argument.
keep.cmd	Logical. Copy cmd.txt per run. Default TRUE.
keep.misc	Logical. Copy any leftover files (e.g. M0_loss_rate.csv) that are neither annual TS nor spatial maps nor cmd.txt. Default TRUE.
overwrite	Logical. If dir.out already exists non-empty, delete it before copying. Default FALSE (function errors out).
ncores	Integer. Number of worker processes to run the per-subdir copy in parallel. Default 1 (sequential). On Windows the outer loop is the bottleneck; setting ncores = 8-16 typically gives near-linear speedup until disk saturates.
long.paths	Logical. On Windows, prefix dir.in/dir.out internally with the \\?\ extended-length path escape so files under long parent paths (e.g. deep OneDrive folders) copy past the 259-char MAX_PATH limit. The user-facing dir.out in the return value is left unprefixed. Ignored on non-Windows. Default TRUE.
verbose	Logical. Print progress + summary. Default TRUE.

Details

What gets copied per run folder:

- Ecospace_Annual_Average_*.csv — all kept.
- Ecospace_Average_*.csv (monthly averages) — dropped.
- EcospaceMap(Biomass|Catch)-<group> <stanza>.csv — kept for groups in groups.
- EcospaceMapEffort-<fleet>.csv — kept for fleets in fleets (exact match on the trailing token, or fleets = "all" for every effort map; NULL drops them all).
- cmd.txt — kept if keep.cmd.
- Anything else (e.g. M0_loss_rate.csv) — kept if keep.misc.

Value

Invisibly, a list with `dir.out`, `n_dirs`, `n_files_in`, `n_files_out`, `bytes_in`, `bytes_out`, and any `warnings` collected (e.g. fleets requested that no run dir contained).

`fn.get_cefi_vars` *List variables and get URLs for CEFI data.*

Description

This function will get information on CEFI data and return a dataframe that can be subset for pulling data.

Usage

```
fn.get_cefi_vars(
  dir.cefi = dir.cefi,
  file.cefivars = "CEFI_vars.csv",
  region = "nwa"
)
```

Arguments

`dir.cefi`	Parent directory for CEFI data.
`file.cefivars`	Name of csv file to be created (or read) containing cefi variables. Default is CEFI_vars.csv.
`region`	CEFI region nwa=northwest Atlantic, nep=northeast Pacific

Examples

```
# example code:
## Not run: vars.cefi <- fn.get_cefi_vars(dir.cefi=dir.cefi, file.cefivars='CEFI_vars.csv', region='nwa')
```

`fn.get_ts_type_table` *Load the Ecosim timeseries type-code lookup table.*

Description

Returns a data frame describing every Ecosim timeseries type code (biomass, catch, mortality, effort, discards, prices, forcing series, etc.), the pool-code role required by each type, and flags indicating whether the series is in absolute units and whether it is a reference (calibration) series. The lookup CSV ships with the package at `inst/extdata/time_series_codes.csv`.

Usage

```
fn.get_ts_type_table()
```

Value

A data frame with columns: `code` (integer type code), `name` (canonical type name, e.g. "Biomass-Rel"), `absolute` (logical), `reference` (logical), and `pool_role` (text describing what the pool code(s) refer to).

`fn.gfisher_default_crosswalk` *Default GFISHER mod-code -> WFS MICE pool-code crosswalk.*

Description

gag ages 0-5+ = mod 20-25 = pc 4-9; red grouper ages 0-5+ = mod 26-31 = pc 10-15. Crosswalk is `pc = mod - 16`.

Usage

```
fn.gfisher_default_crosswalk()
```

Value

data.frame with columns `mod, pc, species`.

`fn.load_gfisher_maxn` *Load GFISHER maxN heatmaps as an observed spatial obs frame.*

Description

Reads the 12 gag + red-grouper maxN `.asc` files, normalises each to unit sum over non-NODATA cells, attaches the WFS MICE pool code via the crosswalk, and records the collection year window (`styear`, `endyear`) alongside each row for year-matched comparison in the objective function.

Usage

```
fn.load_gfisher_maxn(
  gfisher_dir = NULL,
  styear = 2015,
  endyear = 2024,
  crosswalk = NULL,
  variable = "maxn"
)
```

Arguments

`gfisher_dir`	Path to the directory containing the maxN `.asc` files (default: WFS MICE canonical location).
`styear, endyear`	Year window this obs represents. Defaults 2015-2024. Applied to every row unless overridden in the crosswalk.
`crosswalk`	data.frame with columns `mod, pc, species`; see `fn.gfisher_default_crosswalk`. Optionally include a `path` column to point at a specific file; otherwise the file is located by pattern `"_mod<mod>_"`.
`variable`	Character; portion of the filename identifying which variable (default `"maxn"`).

Value

A data.frame with columns `mod, pc, species, path, styear, endyear` and a list-column `map_norm` (each element is a numeric matrix, NA on NODATA cells, unit-sum-normalised).

fn.load_mdat_biomass *Load MDAT interpolated biomass maps as an observed spatial obs frame.*

Description

Attribute-driven loader. Reads the per-map attributes table (which ages each map informs, months, year window, log-space), keeps rows with `use == TRUE` for the requested surveys, locates each `<Species>_<Season>.asc`, back-transforms log-space surfaces, unit-sum normalises over non-NODATA cells, and expands each map's `pcs` into one obs row per pool code. Returns a superset of the columns `fn.spatial_LL` consumes so it drops into the objective fn.

Usage

```
fn.load_mdat_biomass(
  mdat_dir = NULL,
  attributes = NULL,
  surveys = "NEFSC",
  group.names = NULL,
  variable = "Biomass"
)
```

Arguments

`mdat_dir`	Directory containing `<SURVEY>/<Species>_<Season>.asc` subfolders (default: NWACS MICE calibration2 location).
`attributes`	Either a path to the attributes CSV, a data.frame, or NULL (uses `fn.mdat_default_attributes()`).
`surveys`	Character vector of surveys to include (default "NEFSC").
`group.names`	Optional model group.names (indexed by pool code); when supplied the obs `species` label is the group name for that pool code, otherwise it is the MDAT species + season.
`variable`	Character; predicted-map variable name (default "Biomass").

Value

data.frame(survey, season, pc, species, path, styear, endyear, variable) with list-columns `months` (integer vectors) and `map_norm` (numeric matrices, NA on NODATA, unit-sum-normalised).

fn.longvuls *Melt vulnerability matrix*

Description

melt a vulnerability matrix.

Usage

```
fn.longvuls(vuls = vuls)
```

Arguments

vuls	A vulnerability matrix

`fn.M0_loss_rate` *Other-mortality loss rate from Ecospace output.*

Description

Computes the annual M0 loss rate (loss / mean biomass) per group from a pair of Ecospace annual-average output CSVs. Multistanza groups (columns named like "gag 0", "gag 1", ..., "gag 5+") are detected by suffix; for each such base group an additional "<base> total" column is added containing the biomass-weighted aggregate rate: sum(loss) / sum(biomass) across stanzas. This matches the gag/red-grouper Mrt calculations in red tide sims/get red tide mortality S88.R, generalized so the function can be applied to any other-mortality source (red tide, hypoxia, harmful algal bloom, etc.).

Usage

```
fn.M0_loss_rate(
  dir.out,
  file.bio = "Ecospace_Annual_Average_Biomass.csv",
  file.loss = "Ecospace_Annual_Average_OtherMortalityLoss.csv",
  out.file = "M0_loss_rate.csv",
  skip = 31,
  stanza.regex = "\\s+[0-9]+\\+?$",
  write = TRUE
)
```

Arguments

dir.out	Character vector of one or more Ecospace run output directories. Each is expected to contain both file.bio and file.loss.
file.bio	Biomass CSV filename. Defaults to the annual-average file.
file.loss	Other-mortality-loss CSV filename. Defaults to the annual-average file.
out.file	Output CSV filename written into each dir.out when write=TRUE.
skip	Number of metadata header lines to skip when reading data (default 31, matches EwE 6.7 Ecospace output). The function also preserves the blank line after the metadata so the written output mirrors the structure of the source CSV.
stanza.regex	Regular expression matching the stanza-age suffix on a column name. The default "\s+[0-9]+\+?$" matches the EwE convention of "<group> <age>" or "<group> 5+" for terminal stanzas. The base group name is everything before the match.
write	Logical; if TRUE (default), write out.file into each input dir. Header info is copied verbatim from the source biomass CSV, with the Data, line updated to reflect the new content.

Details

Division by zero (biomass = 0) yields NaN/Inf in the returned rate, matching the source mortality scripts; downstream summaries should use na.rm=TRUE in percentile / mean calls.

Value

If `length(dir.out) == 1L`, a numeric matrix `[years x (n_groups + n_stanza_totals)]` of rates only: the source year/time column is dropped from the matrix and its values are carried on `rownames()`. Otherwise a 3-D array `[years x groups x runs]` whose `dimnames[[1]]` are the years/time index and whose 3rd dimension is named by `basename(dir.out)`. (The on-disk CSV written when `write=TRUE` still re-attaches the year column to mirror the source file structure.)

`fn.M0_loss_rate_summary`

Ensemble summary of M0 loss rates (long-form for downstream use).

Description

Reduces an Mrt ensemble to per-year per-group summary statistics (mean + arbitrary percentiles) across an ensemble of model runs. The output is a long-form data frame with one row per (year, group) suitable for plotting (ggplot2) or feeding into stock-assessment projection workflows. Either pass a pre-computed Mrt array (typically from `fn.M0_loss_rate`) via `mrt`, or a vector of model output directories via `dir.out` (the function will call `fn.M0_loss_rate` first; `...` is forwarded).

Usage

```
fn.M0_loss_rate_summary(
  dir.out = NULL,
  mrt = NULL,
  probs = c(0.05, 0.5, 0.95),
  styear = NULL,
  out.file = NULL,
  ...
)
```

Arguments

`dir.out`	Character vector of Ecospace run output directories. Ignored if `mrt` is supplied. Required otherwise.
`mrt`	A 3-D array `[years x groups x runs]` (multi-run) or a 2-D matrix `[years x groups]` (single run) as returned by `fn.M0_loss_rate`. If supplied, `dir.out` is ignored.
`probs`	Numeric vector of percentiles to compute across runs. Default `c(0.05, 0.5, 0.95)` (90\ renamed to `"median"`; other values become `"q05"`, `"q95"`, etc.
`styear`	Optional integer start year. If supplied, the `year` column becomes `styear, styear+1, ...`; otherwise the year labels are taken from `dimnames(mrt)[[1]]` (the row names set by `fn.M0_loss_rate`), falling back to the row index `1..N` if those are absent/non-numeric.
`out.file`	Optional path. If supplied, the long-form summary is written as CSV (one row per year-group combination).
`...`	Forwarded to `fn.M0_loss_rate` when `mrt` is NULL and `dir.out` is supplied.

Value

A data frame with columns `year`, `group`, `mean`, and one column per requested percentile. For a single-run input, the percentile columns equal the mean (no ensemble to summarize across).

`fn.makeparvec` *Construct parameter vectors and bounds for model estimation*

Description

Builds the parameter vector and related metadata (labels, groups, bounds, CVs) for vulnerabilities, environmental responses, dispersal, and mediation parameters. Several objects are assigned in the parent/global environment.

Usage

```
fn.makeparvec(
  do.vuls = TRUE,
  vul_pars = predprey_pars,
  vul.min = 1.01,
  vul.max = 1e+06,
  vul.cv = 0.4,
  vul.bounds.mode = "formula",
  do.env = FALSE,
  env_pars = NULL,
  env.min = 0.25,
  env.max = 2,
  env.cv = 0.2,
  do.disp = TRUE,
  disp_pars = disp_pars,
  disp.cv = 0.2,
  disp.min = 3,
  disp.max = 3000,
  do.med = FALSE,
  med_pars = NULL,
  med.xbase.cv = 0.1,
  do.fltdyn = FALSE,
  fltdyn_pars = NULL,
  fltdyn.min = 0,
  fltdyn.max = 5,
  fltdyn.cv = 0.2,
  fltdyn.bounds.mode = "adaptive",
  do.redtide = FALSE,
  redtide_pars = NULL,
  redtide.min = 0.5,
  redtide.max = 1.5,
  redtide.inf.min = NULL,
  redtide.inf.max = NULL,
  redtide.slope.min = NULL,
  redtide.slope.max = NULL,
  redtide.cv = 0.2
)
```

Arguments

`do.vuls`	Logical; include vulnerability parameters.

vul_pars Data frame of vulnerability parameters.

vul.min, vul.max
Scalar override bounds for the vulnerability search range. Only used when vul.bounds.mode = "override"; ignored under the default "formula" mode.

vul.cv Coefficient of variation for vulnerabilities; controls the log-scale spread of the formula-derived bounds (effective log-sd = exp(2 * vul.cv)).

vul.bounds.mode
Character; either "formula" (default — bounds are log-symmetric around each base.val using vul.cv) or "override" (use [vul.min, vul.max] as the bounds for every vul parameter, ignoring base.val and vul.cv).

do.env Logical; include environmental parameters.

env_pars Data frame of environmental parameters.

env.min Minimum environmental value.

env.max Maximum environmental value.

env.cv Coefficient of variation for environmental parameters.

do.disp Logical; include dispersal parameters.

disp_pars Data frame of dispersal parameters.

disp.cv Coefficient of variation for dispersal parameters.

disp.min Minimum dispersal (unused placeholder).

disp.max Maximum dispersal (unused placeholder).

do.med Logical; include mediation parameters.

med_pars Data frame of mediation shape parameters.

med.xbase.cv CV applied to mediation x-base parameters.

do.fltdyn Logical; include fleet dynamics parameters.

fltdyn_pars Data frame of fleet parameters with at least base.val.

fltdyn.min, fltdyn.max
Safety floor/ceiling for fleet parameters. Under the default "adaptive" bounds mode they are NOT hard limits — the per-parameter bounds widen to base.val ± 4 * fltdyn.cv * base.val so the full normal bell fits inside, while fltdyn.min/fltdyn.max are used as the lower/upper bound only when the bell is tighter than them. Under "scalar" mode every fleet parameter gets these as hard scalar bounds (legacy behavior; will clip the bell if base.val exceeds them).

fltdyn.cv CV for fleet parameter draws; also drives the half-width of the adaptive bounds (4 * fltdyn.cv * base.val).

fltdyn.bounds.mode
Either "adaptive" (default) or "scalar".

do.redtide Logical; include red tide environmental response parameters.

redtide_pars Data frame of red tide response parameters (shape 11 logistic4params).

redtide.min, redtide.max
Scalar multiplier bounds applied to both inflection.adj and slope.adj for red tide responses. Used as the fallback default when the per-parameter overrides below are NULL.

redtide.inf.min, redtide.inf.max
Optional per-parameter bounds for the red tide inflection.adj multiplier (NULL → fall back to redtide.min/max).

redtide.slope.min, redtide.slope.max
Optional per-parameter bounds for the red tide slope.adj multiplier (NULL → fall back to redtide.min/max).

redtide.cv CV used for red tide parameter draws when sampling with a normal distribution.

Value

Invisibly returns NULL. Several objects are set globally.

---

fn.make_aquamaps_shapes

Build AquaMaps trapezoid response shapes from a species list.

Description

Reads a "species list for AquaMaps query" CSV, queries the AquaMaps HSPEN (environmental envelope preference) data, and returns a shapes.out data frame of trapezoid (EwE function type 9) response shapes for depth, temperature, salinity, distance-to-shore, and dissolved oxygen for every species found in AquaMaps. Each shape's four trapezoid parameters come from the species' Min, PrefMin, PrefMax, and Max envelope values for the corresponding variable.

Usage

```
fn.make_aquamaps_shapes(species_csv, db_path = NULL)
```

Arguments

species_csv	Path to the species list CSV. Must contain columns number (functional group number), name (functional group name), and species (scientific name). Any other columns are ignored.
db_path	Optional path to an am.db SQLite file. If NULL (default) the **aquamapsdata** package's standard database location is used; pass a path to use a copy kept elsewhere (see fn.connect_aquamaps_db).

Details

Scientific names in the species list that are not matched in AquaMaps are reported via message() and simply omitted from the output. The first call may prompt to install **aquamapsdata/dplyr** (and verify Rtools) if they are not already present.

Value

A data frame with 11 columns: Function name, Function type, Param 1-Param 6, var, fg num, and source. Parameter columns (Function type, Param 1-6, fg num) are numeric and rounded to 3 decimal places.

See Also

fn.connect_aquamaps_db

Examples

```
## Not run:
# use the package's default database location:
shapes.out <- fn.make_aquamaps_shapes("species list for aquamaps query.csv")

# use an am.db stored on a shared drive:
shapes.out <- fn.make_aquamaps_shapes("species list for aquamaps query.csv",
                                      db_path = "D:/data/aquamaps/am.db")

## End(Not run)
```

`fn.make_cmd_files` *Make command files.*

Description

This function adds taglines to the base command file and saves them in run folders.

Usage

```
fn.make_cmd_files(runlist = runlist, nyrs = nyrs)
```

Arguments

`runlist`	A dataframe where each row is a separate model run. It must include, at a minimum, the 3 variables 'dir.out', which provides the output directory for each run; 'cmd_file' which is the full file path of the command file to be saved in the folder; and 'tag', which is the tagline to be appended to the command file.
`nyrs`	A single number representing the number of years to run ecospace.

Value

Create command .txt files and saves them in their respecitive run folders.

Examples

```
# example code:
## Not run: fn.make_cmd_files(runlist)
```

`fn.make_monthly_climatology_maps`
Make monthly climatology maps.

Description

This stacks all the ascii in stdriver directory and compute long term mean for each month.

Usage

```
fn.make_monthly_climatology_maps(dir.stdriver = dir.stdriver)
```

Arguments

dir.stdriver	Directory for ST driver variable.

Value

Saves 12 ascii files to the variable folder with suffix '9999mm', where mm is numeric month.

Examples

```
## Not run:
# example code:
for(i in 1:length(dirs.stvars)){
fn.make_monthly_climatology_maps(dir.stdriver=dirs.stvars[i])
}
## End(Not run)
```

---

fn.make_static_maps *Make static maps from monthly ST files.*

Description

This stacks all the ascii in stdriver directory and compute long term mean and year-1 mean layers, and saves them as ascii files in the 'static' folder.

Usage

```
fn.make_static_maps(dir.stdriver = dir.stdriver)
```

Arguments

dir.stdriver	Directory for ST driver variable. A 'static' folder will be created one level up if it doesn't exist.

Value

Saves an ascii file to the static folder.

Examples

```
# example code:
## Not run: fn.make_static_maps(dir.stdriver=file.path(dir.stdriver,"tos_cefi"))
```

fn.mdat_default_attributes

Build the default MDAT per-map attributes table.

Description

One row per (survey x species x season) map, seeded from the crosswalk (all ages) and survey metadata (seasons, months, year window, log-space flag). This is the editable control surface: trim `pcs` to the ages a given survey surface actually informs, adjust `months`, and toggle `use`. `pcs` and `months` are comma-separated strings. NEAMAP rows default to `use = FALSE`.

Usage

```
fn.mdat_default_attributes(crosswalk = NULL, survey_meta = NULL)
```

Arguments

`crosswalk`	see `fn.mdat_default_crosswalk`.
`survey_meta`	see `fn.mdat_default_survey_meta`.

Value

data.frame(survey, species, season, use, pcs, months, styear, endyear, logspace).

fn.mdat_default_crosswalk

Default MDAT species -> NWACS MICE pool-code crosswalk.

Description

Maps each MDAT survey species to the NWACS MICE age-structured pool code(s) it informs. A single MDAT biomass surface is replicated across all age pool codes of that species (age-specific spatial obs are not available), e.g. menhaden -> pc 4 (juv) and 5 (adult). `group` is the NWACS group.name (with spaces) used to locate the predicted `EcospaceMap<Var>-<group>.csv`. Pool codes 14-17 (inverts, plankton, detritus) have no MDAT map and are intentionally absent.

Usage

```
fn.mdat_default_crosswalk()
```

Value

data.frame with columns `species` (MDAT filename token), `pc` (NWACS pool code), `group` (NWACS group name).

fn.mdat_default_survey_meta

Per-survey metadata for the MDAT interpolated products.

Description

Year window, available seasons, season->months map, and whether the published surface is in natural-log space (needs back-transform to a biomass pattern before unit-sum normalisation). Reflects the Duke MDAT service contents (checked 2026): NEFSC = raw biomass, Spring+Fall, 2010-2019; NEAMAP = natural-log biomass, Fall only, 2007-2014 (and only a thin nearshore footprint at 15-min). `months` are the calendar months the survey's season maps to, used to average the predicted maps seasonally.

Usage

```
fn.mdat_default_survey_meta()
```

Value

named list keyed by survey name.

fn.mdat_write_attributes

Write the seeded MDAT attributes CSV for the user to edit.

Description

Write the seeded MDAT attributes CSV for the user to edit.

Usage

```
fn.mdat_write_attributes(
  path,
  crosswalk = NULL,
  survey_meta = NULL,
  overwrite = FALSE
)
```

Arguments

`path`	Output CSV path (e.g. `"../obs spatial data/MDAT map attributes.csv"`).
`crosswalk, survey_meta`	passed to `fn.mdat_default_attributes`.
`overwrite`	If FALSE (default) an existing file is left untouched so hand edits are never clobbered.

Value

(invisibly) the path written.

fn.mediation_testval_tags *Make mediation shape parameter taglines for sensitivity runs.*

Description

Create parameter lines for the command file to test sensitivity to mediation shapes.

Usage

```
fn.mediation_testval_tags(meds.base, do.xbase = TRUE)
```

Arguments

meds.base	A dataframe with baseline mediation parameters, this has to be created manually.
do.xbase	Logical. Should the x_base parameter also be tested? (default=TRUE).

Value

A character vector containing the parameter taglines for the command file.

Examples

```
# example code:
## Not run: tags.vul <- fn.vul_testval_tags(predprey=data.frame(pred=1:5,prey=6:10, basevul=2), maxvul=1000,
```

fn.netcdf2ascii_cefi *Create ascii files from CEFI netcdf*

Description

This function processes netcdf files from local machine, downloaded with fn.download_cefi_full_nc(). It peforms spatial subset based on depth map, regrids and resamples to depth, fills missing water cells, and writes ascii files.

Usage

```
fn.netcdf2ascii_cefi(
  nc.files = list.files(path = dir.cefi, pattern = ".nc$", full.names = T),
  depth = depth,
  dir.stdriver = dir.stdriver,
  scalePP = TRUE,
  do.vars = NULL,
  make.monthly = TRUE,
  make.static = TRUE,
  make.climatology = TRUE
)
```

Arguments

nc.files	A character variable containing the file paths of the netcdf files to process.
depth	Raster grid of depth, matching the basemap of Ecospace. Saved ascii files will have the same dimensions.
dir.stdriver	Parent directory for spatial temporal drivers. A folder will be created for each variable in this directory to save the ascii files.
scalePP	Logical. TRUE (default) normalizes primary-production drivers to the mean of the first year of data.
do.vars	Optional character vector of variable-name prefixes to restrict which variables are processed. NULL (default) processes all.
make.monthly	Logical. TRUE will run all the code to create monthly raster stacks. Use FALSE if only want to make static maps.
make.static	Logical. TRUE will create a static map (average) for each variable to use as Ecospace basemap layer.
make.climatology	Logical. TRUE (default) creates a 12-month climatology stack for each variable.

Value

A series of ascii files saved in dir.stdriver directory.

Examples

```
## Not run:
# example code:
files.nc = list.files(dir.cefi, pattern=".nc$", full.names=T)
fn.netcdf2ascii_cefi(nc.files = files.nc, depth=depth.15min,
                     dir.stdriver=file.path(dirname(getwd()),'ST drivers','5min'))
## End(Not run)
```

---

fn.netcdf2ascii_esacci

Create ascii files from ESA CCI netcdf

Description

This function processes ESA CCI satellite netcdf files (e.g. chlor_a, SST, SSS) from the local machine, downloaded with `fn.pull_esacci()`. It loops through monthly timesteps, resamples to the depth grid, fills coastal cells, applies the Morel and Berthon (1989) depth-integration for surface chlorophyll, normalizes PP drivers to the first-year mean, and writes ascii files for input to Ecospace. It can also produce static maps and monthly climatologies.

Usage

```
fn.netcdf2ascii_esacci(
  nc.files = list.files(path = dir.cefi, pattern = ".nc$", full.names = T),
  depth = depth,
  dir.stdriver = dir.stdriver,
  scalePP = TRUE,
```

```
  do.vars = NULL,
  zint.method = c("MB", "MBMML"),
  make.monthly = TRUE,
  make.static = TRUE,
  make.climatology = TRUE
)
```

Arguments

nc.files	A character vector containing the full file paths of the netcdf files to process.
depth	Raster grid of depth, matching the basemap of Ecospace with land cells NA. Saved ascii files will have the same dimensions.
dir.stdriver	Parent directory for spatial temporal drivers. A folder will be created for each variable in this directory to save the ascii files.
scalePP	Logical. TRUE will normalize primary-production drivers (chlor_a and its depth-integrated companion) to the mean of the first year of data. Default TRUE.
do.vars	Optional character vector of variable name prefixes used to filter which subdirectories are processed in the static/climatology step. Default NULL processes all.
zint.method	Character vector of one or both depth-integration methods for surface chlorophyll; each is written to its own tagged _Zint_<method> folder. "MB" uses Morel and Berthon (1989) equations 2b and 2c. "MBMML" integrates the Morel & Maritorena (2001) mean euphotic concentration over the Lee (2007) euphotic (z1%) depth. Default runs both.
make.monthly	Logical. TRUE will create the monthly ST driver ascii files. Default TRUE.
make.static	Logical. TRUE will create a static map (first-year and all-years averages) for each variable. Default TRUE.
make.climatology	Logical. TRUE will create a 12-month climatology stack for each variable. Default TRUE.

Value

A series of ascii files saved in subdirectories of dir.stdriver.

Examples

```
## Not run:
# example code:
nc.files <- list.files(path = dir.esacci, pattern = ".nc$", full.names = TRUE)
fn.netcdf2ascii_esacci(nc.files = nc.files, depth = depth,
               dir.stdriver = file.path(dirname(getwd()), 'ST drivers', '5min', 'ESA_CCI'))
## End(Not run)
```

`fn.netcdf2ascii_glorys` *Create ascii files from GLORYS netcdf*

Description

This function opens a netcdf file, loops through variables and timesteps, and creates ascii files for input to Ecospace. Includes vertical integration of chl-a and coastline filling.

Usage

```
fn.netcdf2ascii_glorys(
  nc.files = list.files(dir.glorys, pattern = ".nc$", full.names = T),
  dir.stdriver = dir.stdriver,
  depth = depth,
  scalePP = TRUE,
  do.vars = NULL,
  make.monthly = TRUE,
  make.static = TRUE,
  make.climatology = TRUE
)
```

Arguments

`nc.files`	A character variable containing the full file paths of the netcdf files to process.
`dir.stdriver`	Parent directory for spatial temporal drivers. A folder will be created for each variable in this directory to save the ascii files.
`depth`	Raster grid of depth, matching the basemap of Ecospacem with land cells NA. Saved ascii files will have the same dimensions.
`scalePP`	Logical. TRUE (default) normalizes primary-production drivers to the mean of the first year of data.
`do.vars`	Optional character vector of variable-name prefixes to restrict which variables are processed. `NULL` (default) processes all.
`make.monthly`	Create the monthly ST driver ascii file. Default TRUE.
`make.static`	Also create the static maps for initialization. Default TRUE.
`make.climatology`	Logical. TRUE (default) creates a 12-month climatology stack for each variable.

Value

A series of ascii files saved in dir.stdriver directory.

Examples

```
# example code:
## Not run:
files.nc = list.files(dir.glorys, pattern=".nc$", full.names=T)
fn.netcdf2ascii_glorys(nc.files = files.nc, depth=depth.15min,
                       dir.stdriver=file.path(dirname(getwd()),'ST drivers','15min'))
## End(Not run)
```

fn.objfxn1 *Ecospace objective function, timeseries only*

Description

Calculates the composite likelihood.

Usage

```
fn.objfxn1(dir.pred, obs.ts = obs.ts, run.idx = 1, fit.abs.catch = TRUE)
```

Arguments

dir.pred	Full directory path to model output folder containing .csv files.
obs.ts	A list object containing reference timeseries information, as that created by fn.read_ecosim_timeseries().
run.idx	If dir.pred has multiple output folders, use this to select which to calculate.
fit.abs.catch	If TRUE (default), do not rescale predicions. If FALSE, predictions were divided by q, where q=mean(pred)/mean(obs)

Value

A vector containing the total and functional group specific likelihood.

fn.parvec2cmd *Write an EwE command file from a parameter vector*

Description

Takes a parameter vector and writes the taglines and Ecospace command file used to launch a single model run.

Usage

```
fn.parvec2cmd(
  par_vec = est_par_vec,
  g = 0,
  idx = 0,
  out_dir = .gen_dir(run_dir, g)
)
```

Arguments

par_vec	Parameter vector to write into the command file.
g	Generation number, used for file tracking in the genetic-algorithm loop. Default 0.
idx	Index for the run identifier. Default 0.
out_dir	Output directory where the command file (and model output) are saved. Defaults to a per-generation subfolder of run_dir.

Value

Path to the written command file.

`fn.plot_ga_pop_responses`

Plot GA population: response-function curves for env/redtide, histograms otherwise.

Description

For each unique parameter group, plots a panel:

- Env response groups (`env<N>_type<S>`) and red-tide groups (`rt<N>_type<S>`): overlay one response curve per population member, with a highlighted curve drawn in blue. By default that curve is the baseline; pass `highlight_pars` to highlight a different parameter set (e.g. the min-LL individual from the final generation).
- All other groups (vul, disp, fleetdyn, med): one histogram per parameter with a vertical dashed line at the highlighted value.

Writes the panels to a multipage PDF.

Usage

```
fn.plot_ga_pop_responses(
  gapop,
  est_par_vec,
  par.groups,
  par.labels,
  env_pars = NULL,
  redtide_pars = NULL,
  file,
  dims = c(3, 3),
  highlight_pars = NULL
)
```

Arguments

`gapop`	Numeric matrix `[popSize x n_pars]` of parameter values.
`est_par_vec`	Named numeric vector of baseline parameter values.
`par.groups`	Character vector mapping each column of `gapop` to a group.
`par.labels`	Character vector of column labels (typically the colnames of `gapop`).
`env_pars`	Data frame of env response baselines (canonical or sensitivity-results schema, see `fn.makeparvec`). May be `NULL`.
`redtide_pars`	Same as `env_pars` for red-tide responses. May be `NULL`.
`file`	Output PDF path.
`dims`	Panel grid `c(rows, cols)`. Default `c(3, 3)`.
`highlight_pars`	Numeric vector of length `ncol(gapop)` drawn in BLUE (dashed vline on histograms; overlaid response curve on env/rt panels). If `NULL` (default — initial-pop plot use case), only the red baseline is drawn. Pass the min-LL individual (e.g. `gapop[which.min(fitness), ]`) for the final-population plot to overlay best-fit on top of the baseline + ensemble.

Value

Invisibly returns `file`.

Markers

RED: `est_par_vec` — baseline / starting values; on env/rt panels the red curve uses adj = 1 (i.e. the natural EwE curve). BLUE: `highlight_pars` (only when supplied) — typically the gen-N best-fit.

`fn.pull_cefi` *Pull CEFI data*

Description

Downloads the full netcdf for MOM6-COBALT variables from https://downloads.psl.noaa.gov/Projects/CEFI/regional_n using the httr::GET function. It applies the spatial subset based on the extent of depth and writes a new netcdf file.

Usage

```
fn.pull_cefi(
  vars.cefi = vars.cefi,
  dir.cefi = dir.cefi,
  reg = "northwest_atlantic",
  depth = depth.5min,
  delete.full.nc = FALSE
)
```

Arguments

`vars.cefi`	A dataframe returned by fn.get_cefi_vars(), likely subset to fewer variables. Must contain the variables "cefi_filename" and "cefi_rel_path".
`dir.cefi`	Parent directory for CEFI data, where the netcdf files will be saved.
`reg`	CEFI region, either northwest_atlantic or northeast_pacific
`depth`	Raster grid of depth, matching the basemap of Ecospace. Saved ascii files will have the same dimensions.
`delete.full.nc`	Logical. TRUE will delete the full nc files if they exist. Default is False.

Examples

```
# example code:
## Not run: fn.pull_cefi(vars.cefi=vars.cefi, dir.cefi=dir.cefi, reg='northwest_atlantic', depth=depth.5min,
```

`fn.pull_esacci` *Download and Process ESA CCI Ocean Data*

Description

Downloads and processes Ocean Colour, Sea Surface Temperature, and Sea Surface Salinity data from ESA Climate Change Initiative (CCI) datasets. Creates spatially and temporally subsetted NetCDF files for use in ocean modeling applications.

Usage

```
fn.pull_esacci(
  dir.out = NULL,
  bbox = NULL,
  startdate = as.Date("1980-01-01"),
  enddate = Sys.Date(),
  oc.url =
   "https://www.oceancolour.org/thredds/dodsC/cci/v6.0-release/geographic/monthly/chlor_a/",
  sss.url =
   "https://data.cci.ceda.ac.uk/thredds/dodsC/esacci.SEASURFACESALINITY.15-days.L4.SSS.multi-sen
  cds_user = NULL,
  cds_key = NULL,
  sst.output_file = "esacci.SST.L4.combined.3_0.monthly",
  chl.output_file = "OC_CCI_chlora",
  sss.output_file = "esacci.SSS.L4.combined.5_5.monthly"
)
```

Arguments

`dir.out`	Character. Output directory path for downloaded netcdf files. Default: dir.esacci
`bbox`	Numeric vector of length 4. Bounding box as c(lon_min, lat_min, lon_max, lat_max)
`startdate`	Date. Start date for data extraction. Default: "1980-01-01"
`enddate`	Date. End date for data extraction. Default: current system date
`oc.url`	Character. Base URL for Ocean Colour CCI data via OPeNDAP
`sss.url`	Character. Complete URL for Sea Surface Salinity CCI data via OPeNDAP
`cds_user`	Character. Copernicus Climate Data Store username
`cds_key`	Character. Copernicus Climate Data Store API key
`sst.output_file`	Character. Base filename for SST output (without extension)
`chl.output_file`	Character. Base filename for the Ocean Colour chlorophyll-a output (without extension). Set `NULL` to skip the chlorophyll download.
`sss.output_file`	Character. Base filename for the Sea Surface Salinity output (without extension). Set `NULL` to skip the salinity download.

Details

This function processes three ESA CCI datasets:

- Ocean Colour: Monthly chlorophyll-a (1997-present)
- Sea Surface Temperature: Monthly SST via Copernicus CDS (any years)
- Sea Surface Salinity: Bi-monthly data aggregated to monthly (2010-2023)

Value

Invisible NULL. Function creates NetCDF files as side effects

Examples

```
## Not run:
# Download data for Gulf of Mexico region
bbox <- c(-98, 18, -80, 31)
fn.pull_esacci(
  bbox = bbox,
  startdate = as.Date("2010-01-01"),
  enddate = as.Date("2020-12-31")
)

## End(Not run)
```

`fn.pull_glorys` *Pull GLORYS*

Description

Pull monthly 1/4 degree physical and biogeochemical hindcast products from the Copernicus marine data store. Requires the Copernicus marine toolbox CLI application. Check website https://data.marine.copernicus.eu for dataset id and variable names.

Usage

```
fn.pull_glorys(
  dir.out = dir.glorys,
  bbox = bbox,
  startdate = startdate,
  enddate = enddate,
  datid.phy = "cmems_mod_glo_phy-all_my_0.25deg_P1M-m",
  datid.bgc = "cmems_mod_glo_bgc_my_0.25deg_P1M-m",
  vars.phy = c("thetao", "so", "bottomT"),
  vars.bgc = c("chl", "nppv", "o2", "phyc"),
  path.copernicusmarine = "C:/~/copernicusmarine.exe",
  getbathygrid = FALSE
)
```

Arguments

dir.out	Output directory to save netcdf files.
bbox	Numeric vector containing spatial domain (W,E,S,N).
startdate	First day to pull data, YYYY-MM-DD.
enddate	Last day to pull data, YYYY-MM-DD.
datid.phy	Dataset id for physical data, visit website.
datid.bgc	Dataset id for biogeochemical data, visit website.
vars.phy	Character vector containing physical variable names to download. Check website for correct names.
vars.bgc	Character vector containing biogeochemical variable names to download. Check website for correct names.
path.copernicusmarine	File path location of copernicusmarine.exe file.
getbathygrid	Logical. Should bathymetric grids be downloaded?

Value

One or more netcdf files containing the data.

Examples

```
# example code:
## Not run: fn.pull_glorys(dir.out=dir.glorys, bbox=bbox, startdate = startdate, enddate = enddate, vars.phy =
```

---

fn.read_ecosim_timeseries

Read an Ecosim timeseries file.

Description

Reads an Ecosim timeseries CSV and parses every series type defined in the type-code lookup (see fn.get_ts_type_table). Handles 4-row headers (Title, Weight, Pool code, Type), 5-row headers (with Pool code 1 / Pool code 2), and the extended 6-row header used for Ecospace runs (adds a "Region" row identifying the Ecospace region per series). The "Type" and optional "Region" rows are auto-detected; the data block is taken to start after the last header row found.

Usage

```
fn.read_ecosim_timeseries(filename, types = NULL, long = FALSE)
```

Arguments

filename	Path to the Ecosim timeseries CSV.
types	Optional subset filter. Either a numeric vector of type codes (e.g. c(0, 1, 6)) or a character vector of canonical type names (e.g. c("BiomassRel", "Catches")). Two shortcuts are also accepted: "reference" (only calibration/reference series) or "forcing" (only forcing series). Default NULL returns everything.
long	Logical. If TRUE, the returned list also includes a tidy long-format data frame ts.long with columns Year, Series, TypeCode, Type, Poolcode, Poolcode2, Weight, Absolute, Reference, Value. Defaults to FALSE.

Value

A list with:

- `ts.head` – enriched header table for all (filtered) series, including the canonical type name and absolute / reference flags.
- `ts` – wide data frame of values (rows = years, columns = series), with column names set to the cleaned series titles.
- `by.type` – named list keyed by canonical type name; each entry is `list(head = ..., data = ...)` containing only the series of that type present in the file.
- `obsB.head`, `obsB`, `obsC.head`, `obsC` – backwards-compatible biomass (codes 0, 1) and catch (codes 6, 61, -6) views. `obsB.head`/`obsC.head` retain the legacy 4-column layout (Title, Weight, Poolcode, Type) so existing callers in `plotting.R` and `Ecospace_objective_functions.R` keep working.
- `ts.long` – only if `long = TRUE`.

Examples

```
## Not run:
ts <- fn.read_ecosim_timeseries("ts_mice_v4_discards.csv")
names(ts$by.type)
ts$by.type$BiomassRel$data
ts_ref <- fn.read_ecosim_timeseries("ts.csv", types = "reference", long = TRUE)

## End(Not run)
```

`fn.read_pred_ecospace_discards_split` *Split Ecospace discards into dead, total, and surviving.*

Description

Derives predicted dead / total / surviving discards from per-region per-fleet-group Catch and Landings CSVs. Uses `DD = Catch - Landings` (dead discards), `DT = DD / Dmort` (total discards, with per-fleet Dmort taken from the type-11 DiscardMortality series in `obs.ts`; a fleet with no type-11 entry defaults to Dmort = 1 so DT = DD), and `DS = DT - DD` (surviving discards). Computed at fleet x group resolution then summed to group level.

Usage

```
fn.read_pred_ecospace_discards_split(
  dir.out,
  timestep,
  regions,
  styear,
  obs.ts,
  fleet.names
)
```

Arguments

dir.out	Path or vector of paths to Ecospace run folders.
timestep	"annual" or "monthly".
regions	Integer vector of region IDs.
styear	Calendar start year used to label the year dimension.
obs.ts	Output of fn.read_ecosim_timeseries, providing the per-fleet type-11 DiscardMortality series.
fleet.names	Character vector indexed by fleet pool code, used to map the "fleet|group" column names to fleet pool codes. If NULL, every fleet defaults to Dmort = 1 (total discards collapse to dead discards).

Value

A list with elements dead, total, surv (each [years, groups, regions, runs]) and the fleet|group versions dead_fg, total_fg, surv_fg; or NULL if Catch/Landings files are missing.

---

fn.read_pred_ecospace_wide

Read Ecospace per-region prediction arrays.

Description

Reads the per-region Ecospace_Annual_Average_Region_<n>_<Var>.csv (or Ecospace_Average_Region_* for monthly) outputs from one or more Ecospace run folders and packs them into a 4-D array [years, groups (or fleet|group), regions, runs]. When aggregate_by_group = TRUE any "fleet|group" columns are summed to the group level.

Usage

```
fn.read_pred_ecospace_wide(
  dir.out,
  varname,
  timestep,
  regions,
  styear,
  aggregate_by_group = FALSE
)
```

Arguments

dir.out	Path or vector of paths to Ecospace run folders.
varname	Variable to read: typically "Biomass", "Catch", or "Landings".
timestep	"annual" or "monthly".
regions	Integer vector of region IDs to read.
styear	Calendar start year used to label the year dimension.
aggregate_by_group	Logical. If TRUE, sum "fleet|group" columns to group level.

Value

A 4-D array `[years, groups, regions, runs]` or `NULL` if no matching files found.

`fn.read_region_areas` *Read region areas for area-weighted multi-region aggregation.*

Description

Ecospace per-region output is a spatially averaged *density* (`"Average regional biomass by group (t/km^2)"`), so a timeseries assigned to several regions must be combined as an area-weighted mean, not a sum or a plain mean. Regions differ enormously in size – in the WFS MICE grid region 12 is a single cell (74 km^2) while region 10 is 734 cells (55,956 km^2) – so an unweighted combination lets one cell carry as much of the predicted signal as several hundred.

Usage

```
fn.read_region_areas(
  path = NULL,
  id.col = "ID",
  area.col = "AREA_KM2",
  land.ids = -9999
)
```

Arguments

`path`	Path to the region attributes CSV. Defaults to the WFS MICE `combined_regions_5min_attributes`
`id.col, area.col`	Column names holding the region ID and its area.
`land.ids`	Region IDs to drop as land / outside the model domain.

Details

Region 0 is the **whole grid** in Ecospace output (`Ecospace_Annual_Average_Region_0_Biomass.csv` carries numerically identical data to the whole-grid file; only the `Data/Area` label rows differ). Its weight is therefore the sum of every non-land region, NOT the attributes row whose ID happens to be 0 – in the WFS MICE table that row is "Water (unsampled)", a genuine sub-region of the grid. Areas are taken in km^2 rather than cell counts so that the latitude-varying size of a 5-min cell is respected.

Value

Named numeric vector of areas, names = region ID as character.

See Also

`fn.ecospace_objfxn`, which consumes this as `region.areas`.

Examples

```
## Not run: region.areas <- fn.read_region_areas()
```

`fn.runEwE` *Run one Ecospace job (system2 version)*

Description

Run one Ecospace job (system2 version)

Usage

```
fn.runEwE(
  cmdfile,
  do.obj = 1,
  console = file.console,
  stdout_target = FALSE,
  stderr_target = FALSE,
  fit.abs.catch = TRUE,
  timeout = 300
)
```

Arguments

`cmdfile`	Character: full path to the command file Ecospace consumes
`do.obj`	Integer: 0=none, 1=`fn.objfxn1`, 2=`fn.objfxn2`, 3=`fn.ecospace_objfxn`
`console`	Character: full path to the Ecospace console executable (defaults to global file.console)
`stdout_target`	NULL/FALSE to suppress, or a file path to capture stdout
`stderr_target`	NULL/FALSE to suppress, or a file path to capture stderr
`fit.abs.catch`	Logical, passed to `fn.ecospace_objfxn` when `do.obj == 3`. If `TRUE` (default), absolute-type series (BiomassAbs, Catches, Landings, Discards, DiscardsTotalAbs) are fit on their literal scale (`q = 1`); if `FALSE`, every series is q-rescaled. Ignored for `do.obj` 1 or 2.
`timeout`	Numeric seconds; passed to `system2`. If the Ecospace child process hasn't exited within this many seconds it is killed and the run is treated as a failure (returns `NA`). Guards against hung EwE.exe processes that would otherwise block a worker indefinitely (e.g., a Windows Error Reporting dialog from a child crash). Default 300 (5 minutes).

Value

Integer exit status (0 = success); invisibly

`fn.runEwE.gapop` *Run Ecospace in parallel, for GA calibration.*

Description

Executes Ecospace runs from a vector of command file paths using parallel workers, retries any failed runs up to a limit, optionally deletes outputs, and returns the fitness (e.g., log-likelihood) vector.

Usage

```
fn.runEwE.gapop(
  files.cmd,
  obj.fxn = 1,
  fit.abs.catch = TRUE,
  delete.output = T,
  max_tries = 50,
  gen = NULL,
 pardist = if (exists("gapop.pardist", envir = .GlobalEnv)) get("gapop.pardist", envir =
    .GlobalEnv) else "uniform",
 vuldist = if (exists("gapop.vuldist", envir = .GlobalEnv)) get("gapop.vuldist", envir =
    .GlobalEnv) else "uniform",
 rtdist = if (exists("gapop.rtdist", envir = .GlobalEnv)) get("gapop.rtdist", envir =
    .GlobalEnv) else "log-uniform",
 fltdist = if (exists("gapop.fltdist", envir = .GlobalEnv)) get("gapop.fltdist", envir =
    .GlobalEnv) else "normal"
)
```

Arguments

`files.cmd`	Character vector of paths to Ecospace command files.
`obj.fxn`	Integer or flag selecting which objective function to use.
`fit.abs.catch`	Logical, passed to the objective function: if `TRUE` (default) absolute-type catch series are fit on their literal scale, otherwise they are q-rescaled.
`delete.output`	Logical; if true, delete generated output folders after runs.
`max_tries`	Maximum number of retry rounds for failed runs.
`gen`	Optional generation index, used only to label output/logs in the genetic-algorithm loop. Default `NULL`.
`pardist`, `vuldist`, `rtdist`, `fltdist`	Sampling-distribution labels for the general, vulnerability, red-tide, and fleet parameters, respectively. Each defaults to the matching `gapop.*dist` global if present, otherwise to a built-in default (`"uniform"`, `"uniform"`, `"log-uniform"`, `"normal"`).

Details

Uses foreach with a parallel backend to run fn_runEwE on each command file. Failed runs (NA results) are identified and resubmitted up to max_tries times. If delete.output is true and run_dir exists in the environment, subdirectories under run_dir are removed after completion.

Value

Numeric vector of fitness values (same length as files.cmd).

`fn.runEwE.parallel` *Run EwE CLI app in parallel*

Description

Calls fn.runEwE in parallel framework.

Usage

```
fn.runEwE.parallel(
  runlist = runlist,
  obj.fxn = 1,
  cl.export = list("runlist", "obs.ts"),
  fit.abs.catch = TRUE,
  timeout = 1800,
  capture_log = TRUE,
  max_retries = 5,
  n_cores = NULL
)
```

Arguments

`runlist`	A dataframe that must contain a variable named 'cmd_file', which is the full file path of the command files to run.
`obj.fxn`	A single number indicating the likelihood function to use. 0=none, 1=timeseries only (`fn.objfxn1`), 2=timeseries and spatial data (`fn.objfxn2`), 3=Ecospace timeseries with fleet-specific landings and discards (`fn.ecospace_objfxn`). `obj.fxn = 3` additionally requires `group.names` and `fleet.names` to be in the worker's environment (add them to `cl.export`).
`cl.export`	A list of objects exported to each worker.
`fit.abs.catch`	Logical, passed to `fn.ecospace_objfxn` when `obj.fxn == 3`. If `TRUE` (default), absolute-type series are fit on their literal scale; if `FALSE`, every series is q-rescaled. Ignored for other `obj.fxn` values.
`timeout`	Per-run wall-clock time limit in seconds; a run exceeding it is abandoned. Default 1800.
`capture_log`	Logical. If `TRUE` (default) each worker's console output is captured to a log file alongside the model output.
`max_retries`	Integer number of times a failed run is retried before it is recorded as an error. Default 5.
`n_cores`	Number of parallel worker cores. `NULL` (default) uses one fewer than the number of detected cores.

Value

Model output is saved according to command file. If do.obj!=0 then a vector likelihoods is returned and added to the runlist dataframe.

Examples

```
# example code:
## Not run: result <- fn.runEwE.parallel(runlist=myrunlist, obj.fxn=1, cl.export=list('myrunlist','obs.ts'))
```

`fn.spatial_LL` *Spatial likelihood component for fn.ecospace_objfxn.*

Description

Per row of `spatial.obs`: builds predicted map for that pool code, averages over the row's own `styear`..`endyear` window, unit-sum normalises both obs and pred on the shared valid mask (non-NA in both), and accumulates cross-entropy $-\sum \tilde{o} \log \tilde{p}$ (multinomial NLL kernel). Returns weighted total plus a per-species named vector.

Usage

```
fn.spatial_LL(
  dir.pred,
  spatial.obs,
  spatial.weight = 1,
  group.names,
  model_styear = 1985
)
```

Arguments

`dir.pred`	Path to an Ecospace output folder (parent of `csv/`).
`spatial.obs`	Output of `fn.load_gfisher_maxn` (or similar), with columns `pc`, `species`, `styear`, `endyear` and list-col `map_norm`.
`spatial.weight`	Numeric scalar multiplier applied to the summed cross entropy (interpretable as a prior precision on the spatial fit).
`group.names`	Character vector indexed by pool code (needed by `fn.ecospace_spatial_pred`).
`model_styear`	Calendar year corresponding to Ecospace step 1 (default 1985).

Value

List with: `total` = weighted total cross-entropy (scalar), `per_species` = named numeric vector, one weighted cross-entropy per row of `spatial.obs`. NA entries indicate rows that could not be scored (missing prediction, empty window, all-NODATA overlap).

fn.STdriver_gif *Make ST driver gifs*

Description

Creates a movie (gif) file from ST driver ascii rasters.

Usage

```
fn.STdriver_gif(
  dir.stdriver = dir.stdriver,
  do.files = NULL,
  datestamps = NULL
)
```

Arguments

dir.stdriver	Full directory path to the ST driver folder.
do.files	A numeric vector indexing which of the ascii files in dir.stdriver to plot. The default NULL plots all files in the folder.
datestamps	Optional character vector of timesteps for each file. Setting to NULL (default) will pull the timestep from the ascii file suffix that is ...YYYYMM.asc.

Value

Place a gif file in a folder one level up from dri.stdriver

Examples

```
#example code:
## Not run: fn.STdriver_gif(dir.stdriver="./ST drivers/30min/wc_vert_int_npp_cefi", do.files=NULL, datestamps
```

fn.STdriver_timeseries *Get ST driver timeseries average.*

Description

Creates a movie (gif) file from ST driver ascii rasters.

Usage

```
fn.STdriver_timeseries(
  dirs.stdriver = dirs.stdriver,
  datestamps = NULL,
  grid.subset = NULL
)
```

Arguments

dirs.stdriver	A character vector containing the full directory paths to 1 or more ST driver variable folders.
datestamps	Optional character vector of timesteps for each file. Setting to NULL (default) will pull the timestep from the ascii file suffix that is ...YYYYMM.asc.
grid.subset	A numeric vector of lenght 4 (xmin,xmax,ymin,ymax) indicating which grid cells subset and then average over. These are row and column indexes. NULL, the default, averages over the entire spatial grid. A value of c(1,3,1,3) would get the top left 3x3 corner. Need to switch to a grid mask so cells subset can be any shape/cells.

Value

Saves a STdriver_timeseries.csv file in the parent directoryr

Examples

```
#example code:
## Not run: fn.STdriver_timeseries(dirs.stdriver=list.dirs("./ST drivers/30min")[1:2], grid.subset=NULL, date
```

---

fn.vul_testval_tags *Make vulnerability parameter taglines for sensitivity runs.*

Description

Create parameter taglines for the command file to test sensitivity of vulnerability parameters.

Usage

```
fn.vul_testval_tags(
  predprey,
  pct.change = 0.5,
  maxvul = 100,
  minvul = 1.01,
  is.maxvul.mult = T,
  test.type = "absolute"
)
```

Arguments

predprey	A dataframe containing 3 numeric columns, "pred", "prey", and "basevul", containing the group numbers of predator prey pairs to evaluate and starting values for the vulnerabilities. Missing (NA) in the prey column will set the vulnerability by predator column.
pct.change	The percent change to use when test.type='percentage'. Applied as pct.change x base and (1+pct.change) x base. Default is 0.5.
maxvul	A high vulnerability number to test
minvul	A low vulnerability number to test.
is.maxvul.mult	Logical indicating whether the maxvul is treated as a multiplier on the basevul (TRUE) or as the absolute value (FALSE).

`test.type` Either 'absolute' in which case maxvul, minvul, and maxvul.mult are applied, or 'percentage', where pct.change and (1+pct.change) is multiplied by the base value.

Value

A character vector containing the parameter taglines for the command file.

Examples

```
# example code:
## Not run: tags.vul <- fn.vul_testval_tags(predprey=data.frame(pred=1:5,prey=6:10, basevul=2), maxvul=1000,
```

`fn.weighted_mrt_summary` *Weighted Mrt ensemble summary*

Description

Long-form ensemble summary of per-run Mrt using arbitrary per-run weights (e.g. Akaike-style weights derived from each individual's LL). Mirrors `fn.M0_loss_rate_summary` but replaces the plain ensemble mean and quantiles with weighted versions, so individuals with better fit contribute more to the band.

Weighted mean per (year, group): `sum(x_i * w_i) / sum(w_i)` (NA-safe; runs with NA at that cell drop). Weighted quantiles per (year, group): linear interpolation on the empirical weighted CDF (`cumsum(w_sorted) / sum(w_sorted)`).

Usage

```
fn.weighted_mrt_summary(
  mrt,
  weights,
  probs = c(0.05, 0.5, 0.95),
  styear = NULL,
  out.file = NULL
)
```

Arguments

`mrt` 3-D array `[years x groups x runs]` of per-run Mrt values (typically built by stacking outputs of `fn.M0_loss_rate`). A 2-D `[years x groups]` matrix is also accepted (degenerate single-run case).

`weights` Numeric vector of length `dim(mrt)[3]` (one weight per run). Non-positive and NA weights are dropped before summarizing. The function normalizes the remaining weights internally; you do not need to normalize beforehand.

`probs` Numeric vector of percentiles in `[0, 1]`. Default `c(0.05, 0.5, 0.95)`.

`styear` Optional integer start year. If supplied, the `year` column becomes `styear, styear+1, ...`; otherwise the year labels come from `dimnames(mrt)[[1]]` (see `fn.M0_loss_rate`).

`out.file` Optional path; if supplied, the long-form summary is written to CSV.

Value

Data frame with columns `year`, `group`, `weighted_mean`, then one column per requested percentile (`q05`, `median`, `q95`, ...).

Examples

```
## Not run:
# Stack per-run Mrt into a 3-D array
mrt <- abind::abind(lapply(final_pop_dirs, fn.M0_loss_rate, write = FALSE),
                    along = 3)

# Akaike-style weights from GA fitness vector
w <- fn.akaike_weights(fitness_g20, scale = 2)

fn.weighted_mrt_summary(mrt, weights = w,
                        probs = c(0.05, 0.5, 0.95),
                        out.file = "mrt_weighted_summary.csv")

## End(Not run)
```

`fn_fleetdyn_testval_tags` *Make fleet dynamics parameter taglines for sensitivity runs.*

Description

Create parameter lines for the command file to test sensitivity of effective power and effort multiplier parameters.

Usage

```
fn_fleetdyn_testval_tags(fleetdyn.base, pct.change = 0.5)
```

Arguments

`fleetdyn.base`	A dataframe with baseline parameters, exported from EwE and read in the setup file.
`pct.change`	The percent change to use, applied as pct.change x base and (1+pct.change) x base. Default is 0.5 (+/-50%).

Value

A character vector containing the parameter taglines for the command file.

Examples

```
# example code:
## Not run: tags.vul <- fn.vul_testval_tags(predprey=data.frame(pred=1:5,prey=6:10, basevul=2), maxvul=1000,
```

mutate — *Mutate population with parameter-wise random resets*

Description

For each gene of each individual, with probability `mutation_rate` redraw the value within parameter-specific bounds. The vulnerability bounds adapt to the current population mean (so the search drifts up or down with the population) and vul and red tide draws are log-uniform to match the initial-population sampling philosophy. All other parameters use fixed `L.bounds`/`U.bounds` with linear-uniform draws.

Usage

```
mutate(population)
```

Arguments

`population`	Matrix of individuals (rows = individuals, columns = parameters).

Details

Adaptive vul bounds: `Gamma(shape=2, mean=pop_mean)`; bounds are the 1\ 99\ mean upward (or downward), so does the mutation search window. Vul draws inside those bounds are log-uniform.

Red tide draws are log-uniform over the fixed `[L.bounds, U.bounds]` for `redtide.par.idx` (symmetric around the baseline of 1 in log space, matching `fn.GApop(rtdist='log-uniform')`).

Requires global variables: `n_pars`, `mutation_rate`, `vul.par.idx`, `redtide.par.idx`, `par.labels`, `L.bounds`, `U.bounds`.

Value

Mutated population matrix (same dimensions as `population`).

select_parents — *Select parents by rank-based sampling (minimization)*

Description

Ranks individuals by fitness for a MINIMIZATION objective (lower = better), converts ranks to selection probabilities, and samples a new parent population with replacement. The lowest-fitness individual receives the highest rank and therefore the highest probability of being chosen.

Usage

```
select_parents(gapop, fitness)
```

Arguments

`gapop`	Matrix or data frame of individuals (rows = individuals, columns = parameters).
`fitness`	Numeric vector of fitness values (length equals nrow(gapop)); lower is better.

Details

Ranks are computed via `rank(-fitness, ties.method='random')` so the smallest fitness gets rank `nrow(gapop)` (i.e., the highest selection weight). Selection pressure is roughly linear: the best individual is only ~2x more likely than the median to be picked. Use `tournament_select` for stronger pressure.

Value

Matrix of selected parents with the same dimensions as `gapop`.

`tournament_select` *Select parents by tournament (minimization)*

Description

Tournament selection for a MINIMIZATION objective. For each parent slot, draw `k` individuals at random (without replacement) and keep the one with the lowest fitness. Stronger and more tunable selection pressure than rank-based sampling: larger `k` pushes harder toward the current best, `k=2` is near-rank-equivalent.

Usage

```
tournament_select(gapop, fitness, k = 3)
```

Arguments

`gapop`	Matrix or data frame of individuals (rows = individuals, columns = parameters).
`fitness`	Numeric vector of fitness values (lower is better). NA fitnesses are treated as worst (lose every tournament unless all contenders are NA, in which case a random contender wins).
`k`	Tournament size. Default 3.

Value

Matrix of selected parents with the same dimensions as `gapop`.

Index